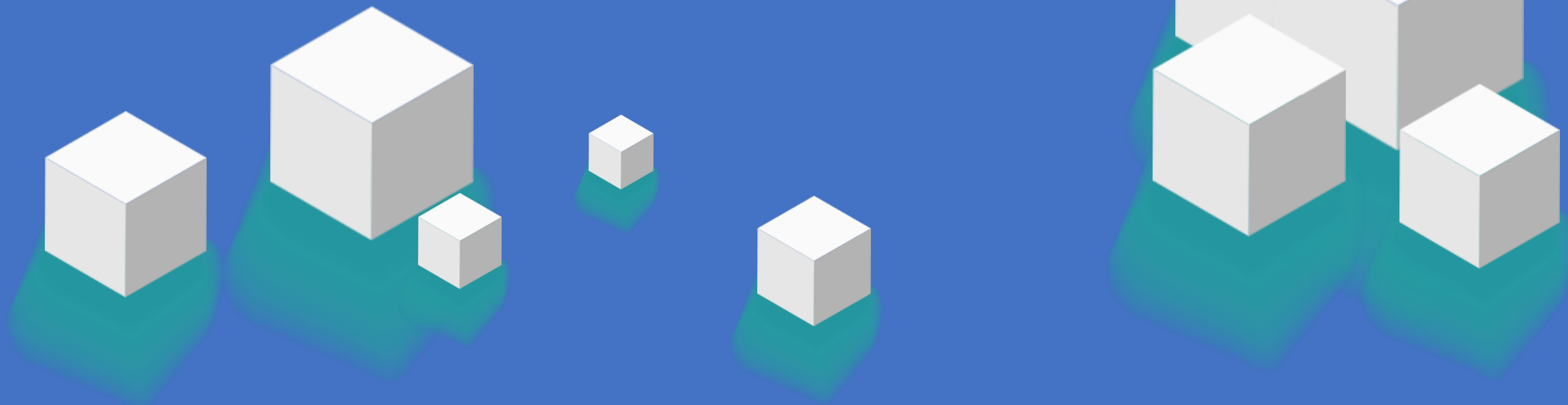


AI服务核心：

高并发原理与性能监控调优



>> 今天的学习目标

AI服务核心：高并发原理与性能监控调优

- 高并发 AI 服务的内存瓶颈

什么是 KV Cache

KV Cache 的瓶颈

PagedAttention机制

- vLLM架构实现详解

Batch的概念

静态 Batch vs Continuous Batching

vLLM 的 Iteration-level Scheduling

PagedAttention 在 vLLM 中的实现

- 推测解码

推测解码算法

实验结果

- MEDUSA推理框架

- EAGLE推理框架

草稿阶段

验证阶段

- vLLM 性能监控与调优

高并发 AI 服务的内存瓶颈

什么是 KV Cache

Thinking: 什么是 KV Cache?

在 Transformer 的自回归生成过程中，为避免重复计算，将每一层的 Key（K）和 Value（V）张量缓存起来。

假设生成句子 "我爱中国"，输入序列长度为 4：

步骤	输入	无 KV Cache	有 KV Cache
Step 1	[我]	计算 Q1, K1, V1 → 生成 "爱"	计算 Q1, K1, V1 → 缓存 K1, V1
Step 2	[我, 爱]	重新计算 K1, V1（重复!） 再计算 K2, V2	复用缓存的 K1, V1 只计算新的 K2, V2
Step 3	[我, 爱, 中]	重新计算 K1, K2, V1, V2	复用 K1..2, V1..2，只算 K3, V3

什么是 KV Cache

显存占用计算 -- 以 LLaMA-2-7B 为例

- 模型参数：隐藏维度 4096，层数 32，注意力头数 32，FP16 精度
- 每 token 的 KV Cache = $2 (K+V) \times 32 (\text{层}) \times 4096 (\text{维度}) \times 2 (\text{FP16字节}) = 524,288 \text{ 字节} = 0.5 \text{ MB/token}$
- 上下文 4096 tokens = $4096 \times 0.5 \text{ MB} = 2 \text{ GB/请求}$
- 并发 100 个请求 = $100 \times 2 \text{ GB} = 200 \text{ GB} \gg \text{A100 80GB}$

关键瓶颈：200 GB 的 KV Cache 远超单卡 A100（80GB）的显存容量。即便模型本身只占 14GB，KV Cache 才是高并发场景下真正的显存杀手。

KV Cache 的瓶颈

瓶颈 1: 内存碎片

内部碎片：为每个请求预分配最大长度（如 2048），实际只用 500，剩余 1548 被浪费

外部碎片：请求 A 占用 0-500，请求 B 占用 1000-1500，中间 500-999 空闲但无法分配给需要连续内存的请求 C

显存布局（时间轴）：

T1: [请求A: 2048] [请求B: 2048] [请求C: 2048] ...

↓ 请求 A 完成，释放空间

T2: [空闲: 2048] [请求B: 2048] [请求C: 2048] ...

↓ 新请求 D 需要 4096，虽然总空闲 2048，但不连续，无法分配！

KV Cache 的瓶颈

瓶颈 2: 动态长度导致显存浪费

比如场景:

- 用户问: "你好" → 生成长度 100 tokens
- 用户问: "解释量子力学" → 生成长度 2000 tokens
- 传统方案按最大长度预分配 → 平均利用率 < 30%

用户问 "你好"
实际 100 tokens

预分配剩余
浪费 1948 tokens

用户问 "解释量子力学"
实际 2000 tokens

48

同一系统中不同请求的实际长度差异

KV Cache 的瓶颈

瓶颈 3: Copy-on-Decoding 开销

当使用 Beam Search（束搜索）时，需要将 KV Cache 完整复制到每个 beam 分支中：

- 每次复制量 = Beam Width(4) x 序列长度(1000) x 0.5MB = 2 GB 数据拷贝
- 触发频率 = 每生成一个 token 都要发生

=> 严重拖慢生成速度

PagedAttention

PagedAttention:

核心思想：块化存储,类似操作系统虚拟内存：

- 虚拟内存：将不连续的物理页映射给连续的虚拟地址
- PagedAttention：将不连续的显存块映射给逻辑上连续的序列

=> 不再要求 KV Cache 在物理显存中连续存储，而是分割成固定大小的 Block（通常 16 tokens/块）

通过 Block Table 维护逻辑到物理的映射关系。

PagedAttention 存储映射示意

逻辑视图（连续的序列）



Block Table（映射表）

Block ID	物理地址
0	0x1000
1	0x5000

物理显存（非连续存储）



就像图书馆的书架：一套丛书不必摆在连续的格子里，只要在目录卡片（Block Table）上记录每一册放在哪个架子的哪一层，读者照样能按顺序阅读。

=> PagedAttention 就是 GPU 显存的 图书馆目录卡片。

PagedAttention机制1： 按需分配

机制 1： 按需分配（On-demand Allocation）

- 传统方式：

启动时预分配 $\text{max_seq_len} (2048) \times 0.5\text{MB} = 1024 \text{ MB}$

无论实际用多少，全部占满

- PagedAttention

生成 1 个 token 才分配 1 个 block（16 tokens 容量）

用满一个 block 才申请新 block

Thinking： 生成 100 tokens 的序列，使用 PagedAttention 会节省多少显存？

- 需要 blocks = $\text{ceil}(100 / 16) = 7$ 个 blocks

- 实际占用 = $7 \times 16 \times 0.5\text{MB} = 56 \text{ MB}$

- 传统方式 = 1024 MB

- 节省 = 94.5% 显存

7 个 block，每个 block 能容纳 16 个 token，每个 token 的 KV Cache 占 0.5MB。计算的是分配的容量（7 个 block 的总容量=112 个 token 的空间）

PagedAttention机制2：写时复制

机制 2：写时复制（Copy-on-Write）

解决 Beam Search / Parallel Sampling 的显存爆炸问题。

比如：Parallel Sampling 场景对 "解释AI" 进行 3 个并行采样

- Prompt 阶段：三个采样共享同一份 KV Cache



引用计数 = 3（三个 Sample 共享，物理存储只有一份）

PagedAttention机制2：写时复制

- 生成阶段：分歧点才分配私有 Block



PagedAttention: 5 blocks (共享) + 3 blocks (私有) = 8 blocks

传统方式: $3 \times 6 = 18$ blocks (完整复制 3 份)

=>节省 55% 显存

Beam Search

Beam Search（束搜索）

核心思想：生成时不只保留 当前最优 的那一条路，而是同时保留 k 条最有可能的候选路径，最后选最好的。

比如：翻译 "I love you", beam width = 3

Step 1: 第一个词有 3 个候选

路径 A: “我”，路径 B: “吾”，路径 C: “俺”

Step 2: 每条路径各自扩展，再保留总分最高的 3 条

A1: “我爱”，A2: “我喜欢”

B1: “吾爱”

问题：3 条路径的前缀部分（如 "I love" 对应的 KV Cache）是相同的，但传统方式要复制 3 份！

Parallel Sampling

Parallel Sampling（并行采样）

核心思想：对同一个 **prompt**，同时生成多个不同的回复（每次随机采样不同），让用户选最好的。

比如：用户问 "写一首关于春天的诗"，同时生成 3 个版本

同一个 **Prompt**，3 个采样：

回复 1: "春风拂面柳丝长..."

回复 2: "三月桃花开满枝..."

回复 3: "细雨润物无声处..."

问题：3 个回复的 **Prompt** 部分（"写一首关于春天的诗"）完全相同，**KV Cache** 也相同，但传统方式要存 3 份！

Beam Search 与 **Parallel Sampling**，多条路径/多个回复共享同一个 **Prompt** 前缀，**KV Cache** 完全相同。传统做法大量显存浪费。**Copy-on-Write** 共享部分只存一份，只有在产生分歧（不同的输出 **token**）时才分配新的内存。

PagedAttention机制3：动态重排

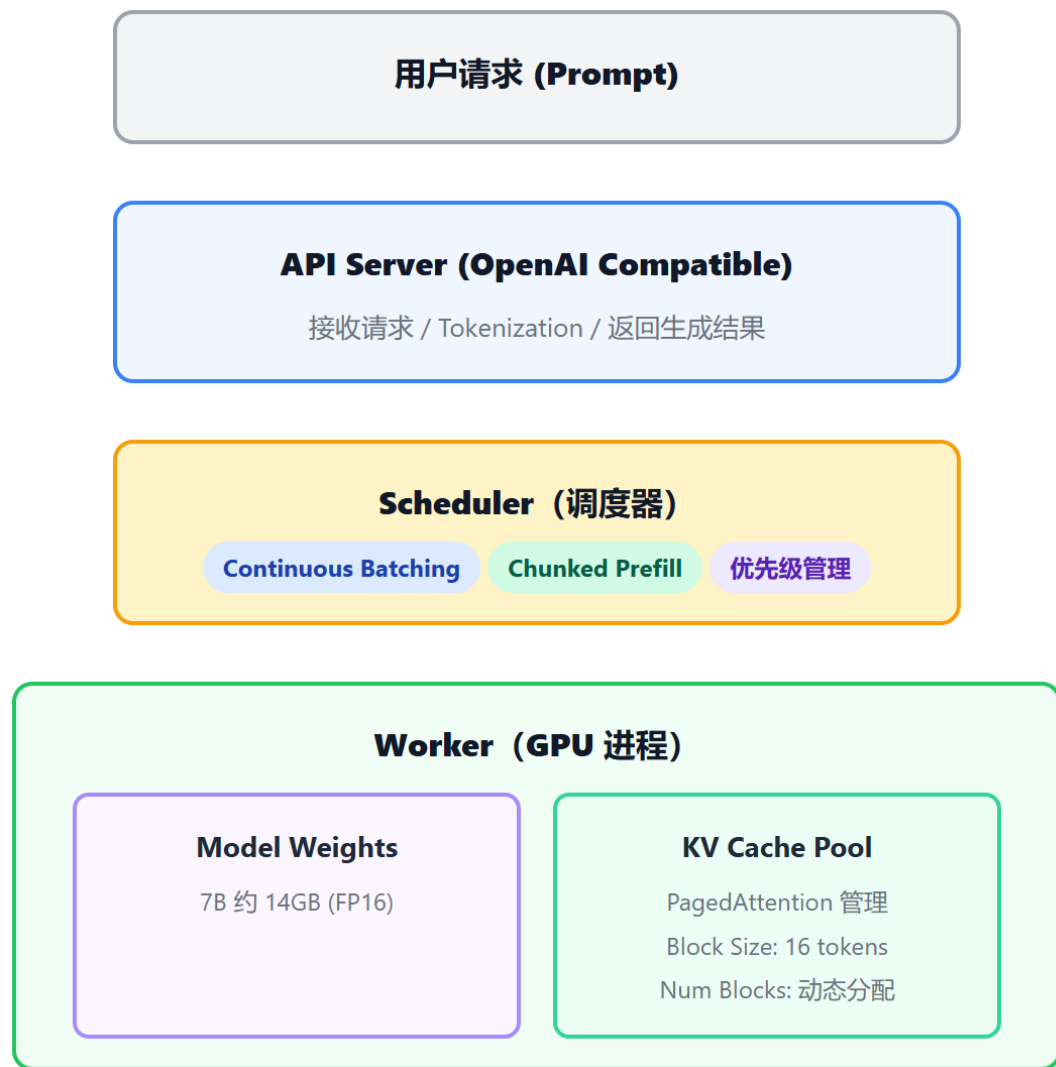
- Continuous Batching 的关键

当新请求到达时，无需寻找连续的大块显存，只需收集分散的空闲 **blocks** 即可。

就像停车场 -- 不需要找一排连续的车位，有空位就能停。

vLLM架构实现详解

vLLM系统架构



Batch的概念

Thinking: Batch 是什么?

Batch（批处理）指的是把多个用户请求打包在一起，送入 GPU 做一次前向传播。

为什么要 Batch? GPU 擅长并行计算。处理 1 个请求和同时处理 8 个请求，GPU 耗时差不多（计算单元足够多），但吞吐量翻了 8 倍。

一辆公交车，载 1 个人和载 30 个人跑一趟的油耗差不多，但运力完全不同。

假设同时有 3 个用户发来请求：

- 用户 A 问 "你好" -- 需要生成 100 个 token
- 用户 B 问 "解释量子力学" -- 需要生成 200 个 token
- 用户 C 问 "写个排序算法" -- 需要生成 150 个 token

把这 3 个请求放进同一个 Batch，GPU 每一步（iteration）同时为 3 个请求各生成 1 个 token。

=> 问题在于：它们生成长度不同，短的完成了长的还没完，怎么办?

静态 Batch vs Continuous Batching

静态 Batch（同步等待）

Batch 中所有请求必须等待最长的那个完成：

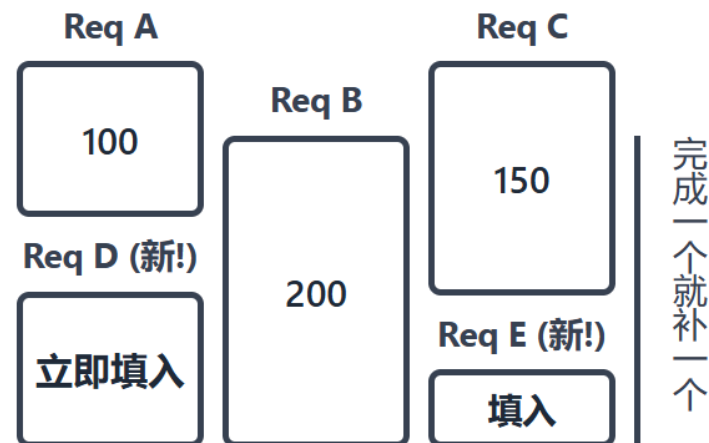


等待时间 = $\max(100, 200, 150) = 200$ steps

Req A 完成后 GPU 对应位置空闲 100 steps，白白浪费

Continuous Batching（动态填充）

Req A 完成后，立即从队列取新请求填入：



GPU 永远满负荷运行，没有空闲浪费

请求完成即刻腾出位置给新请求

vLLM 的 Iteration-level Scheduling (伪代码)

vLLM 的 Iteration-level Scheduling (伪代码)

```
# 每个 iteration (生成一个 token) 调度一次
while True:
    # 1. 处理新到达的请求 (Prefill 阶段)
    new_requests = get_new_requests()
    for req in new_requests:
        blocks_needed = ceil(len(req.prompt_tokens) / block_size)
        allocate_blocks(req, blocks_needed)
        batch.add(req)

    # 2. 执行一次前向传播 (每个请求生成一个 token)
    outputs = model_forward(batch)
```

```
# 3. 处理完成的请求
for req in batch:
    if outputs[req].token == EOS:
        batch.remove(req)
        free_blocks(req) # PagedAttention 立即释放 blocks
        # 关键: 本 iteration 即可加入新请求!
```

Thinking: 为什么叫 Iteration-level?

传统方案是 Request-level: 整个 batch 的所有请求都完成才调度下一批。而 vLLM 在每生成一个 token 后就检查和调度, 粒度细到单次迭代。

=> 餐厅不等所有客人吃完才翻台, 而是哪桌吃完就立刻安排新客人。

Summary

性能对比

指标	静态 Batch	Continuous Batching	提升
GPU 利用率	40-60%	85-95%	1.5-2x
吞吐量 (tokens/s)	800	1800	2.25x
平均延迟 (P50)	1200ms	800ms	1.5x

PagedAttention 在 vLLM 中的实现细节

Block Table 数据结构

```
class BlockTable:
    def __init__(self, block_size=16):
        self.block_size = block_size
        self.blocks = []      # 物理块索引列表
        self.num_tokens = 0   # 逻辑 token 数量

    def append_token(self):
        if self.num_tokens % self.block_size == 0:
            # 当前 block 满了, 需要新块
            new_block = gpu_memory.allocate_block()
            self.blocks.append(new_block)
        self.num_tokens += 1
```

```
def get_physical_indices(self, position: int):
    block_idx = position // self.block_size
    offset = position % self.block_size
    physical_block = self.blocks[block_idx]
    return physical_block * self.block_size + offset
```

PagedAttention 在 vLLM 中的实现细节

Attention 计算时的 Block 映射（CUDA Kernel 伪代码）

```
# 传统 Attention: Q @ K.T, K 是连续的 [seq_len, head_dim]
# PagedAttention: K 是分散的, 需要自定义 CUDA Kernel

for each token i in sequence:
    block_idx = i // block_size
    offset = i % block_size
    physical_block = block_table[block_idx]
    K_i = load_from_physical_block(physical_block, offset)
    score = Q[token] @ K_i.T
```

性能优化关键:

vLLM 的 CUDA Kernel 在 GPU 上并行处理多个序列的分散内存访问, 通过合并内存请求减少延迟。

=> GPU 的多个线程同时访问不同 block 的数据, 把分散的小请求合并成少量大请求, 充分利用显存带宽。

推测解码

核心思想

推测解码（又称投机采样）来自论文《Fast Inference from Transformers via Speculative Decoding, 2023》。

核心角色：

- Target Model（目标模型）：待加速的大模型（如 70B）
- Draft Model（草稿模型/近似模型）：用来帮助加速的小模型（如 7B）

=> 用 Target Model 只推理 9 次，就解码出 38 个 token，推理速度获得较大提升。

推测解码(Speculative Decoding)

Speculative Decoding: 推测解码, 投机采样

Fast Inference from Transformers via Speculative Decoding,
2023 <https://arxiv.org/abs/2211.17192>

用 target model (目标模型) 指代待加速的大模型

用 approximation model (近似模型) 指代用来帮助加速大模型的小模型。

在这个例子中, target模型只推理了9次, 就解码出了38个token, 推理速度获得了较大提升。

[START] japan ' s benchmark bond n
[START] japan ' s benchmark nikkei 22 5
[START] japan ' s benchmark nikkei 225 index rose 22 6
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 points
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 points , or 0 1
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 points , or 1 . 5 percent , to 10 , 98
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 points , or 1 . 5 percent , to 10 , 989 . 79 in
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 points , or 1 . 5 percent , to 10 , 989 . 79 in tokyo late
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 points , or 1 . 5 percent , to 10 , 989 . 79 in late morning trading . [E

绿色token: 近似模型提出且目标模型接受的建议; 红色token: 近似模型提出但目标模型拒绝的建议;

蓝色token: 目标模型对于红色token的订正;

推测解码(Speculative Decoding)

推测解码算法：

位置： 1 2 3 4

输入： 我 爱 中 国

输出： 爱 中 国 EOS

Thinking: 如果是自回归模型，只能一个一个输出：

Step1: 输入“我”，输出“爱”；

Step2: 输入“我爱”，输出“中”；

Step3: 输入“我爱中”，输出“国”；

Step4: 输入“我爱中国”，输出“EOS”；

是否有推测解码，只要输入“我”，就会输出“爱 中国 EOS”四个token作为草稿

然后用这四个draft token一起放到原来的模型，跑一下各个位置的输出，进行验证

推测解码 (Speculative Decoding)

假设实际结果为：

位置： 1 2 3 4

输入： 我 爱 中 华

输出： 爱 中 国 EOS

假设预测给出：“爱”和“中”命中了，但是“华”没对。

=> 这种情况下，跑一次模型也能解码出两个token，推理效率依然有提升。

有时候预测结果也可能完全跑偏，比如给它输入“我”，它输出“叫小明”，这就和原模型一个都没对上。

=> 只要统计上，预测给出的草稿平均命中token数 > 0 ，我们就有机会获得推理加速。

speculative decoding的主要思路：就是一个规模比较小的模型（approximation model或draft model），而我们想要加速的原模型则叫target model。

推测解码 (Speculative Decoding)

M_p 是target model

M_q 是draft model

prefix是当前的输入

Step1. 生成 draft tokens

Step2. 评分 draft tokens

Step3. 判断是否接受

Step4. 处理接受/拒绝结果

Step5. 返回结果

Algorithm 1 SpeculativeDecodingStep

Inputs: $M_p, M_q, prefix$.

▷ Sample γ guesses $x_1, \dots, x_\gamma$ from M_q autoregressively.

for $i = 1$ **to** γ **do**

$q_i(x) \leftarrow M_q(prefix + [x_1, \dots, x_{i-1}])$

$x_i \sim q_i(x)$

end for

▷ Run M_p in parallel.

$p_1(x), \dots, p_{\gamma+1}(x) \leftarrow$

$M_p(prefix), \dots, M_p(prefix + [x_1, \dots, x_\gamma])$

▷ Determine the number of accepted guesses n .

$r_1 \sim U(0, 1), \dots, r_\gamma \sim U(0, 1)$

$n \leftarrow \min(\{i - 1 \mid 1 \leq i \leq \gamma, r_i > \frac{p_i(x)}{q_i(x)}\} \cup \{\gamma\})$

▷ Adjust the distribution from M_p if needed.

$p'(x) \leftarrow p_{n+1}(x)$

if $n < \gamma$ **then**

$p'(x) \leftarrow \text{norm}(\max(0, p_{n+1}(x) - q_{n+1}(x)))$

end if

▷ Return one token from M_p , and n tokens from M_q .

$t \sim p'(x)$

return $prefix + [x_1, \dots, x_n, t]$

推测解码 (Speculative Decoding)

Step1. 生成 draft tokens

- 使用一个较小的自回归模型(draft model)生成一个长度为K的初步标记序列(draft tokens)。
- 记录这K个草稿标记对应的概率值 p 。

Step2. 评分 draft tokens

- 利用目标大模型(target model)对这K个草稿标记进行评分,获得概率值 q 。
- 这一步通常只需要进行一次前向传播,评分速度快(评分时间与评分单个标记的时间相当)

Step3. 判断是否接受

- 对于每个草稿标记,计算 $\min(1, q/p)$ 作为接受该标记的概率。
- 生成K个 $[0,1]$ 范围内的均匀随机数。
- 如果随机数小于等于 $\min(1, q/p)$,则接受该草稿标记,否则拒绝。

Step4. 处理接受/拒绝结果

- 如果所有K个草稿标记都被接受,从目标模型直接采样第K+1个标记。
- 如果第 t 个草稿标记被拒绝,则:
从 $q(x) - p(x) \geq 0$ 的修正概率分布中采样一个新标记。
将之前接受的草稿标记和新采样的标记连接作为最终结果。

Step5. 返回结果, 即最终生成的标记序列。

实验结果

实验设置:

两个任务上测试了T5版本的标准编码器-解码器模型:

- 1) 在WMT EnDe上微调的英语到德语翻译
- 2) 在CCN/DM上微调的文本摘要

对于这两个任务, 使用了T5-XXL (11B) 作为Mp。

对于近似模型Mq, 测试了几种现有的配置, 即T5-large(800M)、T5-base(250M)和T5-small(77M)。

单个TPU-v4上, 对于批量大小为1的情况, 测量了Wallclock的改进, 分别使用了argmax采样 (温度=0) 和标准采样 (温度=1)。

TASK	M_q	TEMP	γ	α	SPEED
ENDE	T5-SMALL ★	0	7	0.75	3.4X
ENDE	T5-BASE	0	7	0.8	2.8X
ENDE	T5-LARGE	0	7	0.82	1.7X
ENDE	T5-SMALL ★	1	7	0.62	2.6X
ENDE	T5-BASE	1	5	0.68	2.4X
ENDE	T5-LARGE	1	3	0.71	1.4X
CNNDM	T5-SMALL ★	0	5	0.65	3.1X
CNNDM	T5-BASE	0	5	0.73	3.0X
CNNDM	T5-LARGE	0	3	0.74	2.2X
CNNDM	T5-SMALL ★	1	5	0.53	2.3X
CNNDM	T5-BASE	1	3	0.55	2.2X
CNNDM	T5-LARGE	1	3	0.56	1.7X

T5- XXL 11B模型的加速推理结果

实验结果

γ (Gamma):

γ 表示每次迭代中，由近似模型生成的候选令牌的数量。

=> 人工给定的参数

α (Alpha):

α 表示近似模型（ M_q ）生成的候选令牌被目标模型（ M_p ）接受的概率。

α 的值越高，表示近似模型生成的候选项越有可能被目标模型接受。

=> 自动计算出来的结果，不是人工给定

TASK	M_q	TEMP	γ	α	SPEED
ENDE	T5-SMALL ★	0	7	0.75	3.4X
ENDE	T5-BASE	0	7	0.8	2.8X
ENDE	T5-LARGE	0	7	0.82	1.7X
ENDE	T5-SMALL ★	1	7	0.62	2.6X
ENDE	T5-BASE	1	5	0.68	2.4X
ENDE	T5-LARGE	1	3	0.71	1.4X
CNNDM	T5-SMALL ★	0	5	0.65	3.1X
CNNDM	T5-BASE	0	5	0.73	3.0X
CNNDM	T5-LARGE	0	3	0.74	2.2X
CNNDM	T5-SMALL ★	1	5	0.53	2.3X
CNNDM	T5-BASE	1	3	0.55	2.2X
CNNDM	T5-LARGE	1	3	0.56	1.7X

T5- XXL 11B模型的加速推理结果

实验结果

Thinking: 你能从实验结果中，看到什么结论？

比目标模型小几个数量级的近似模型倾向于产生0.5到0.9之间的 α 值。

对于所有模型，调整后的分布越尖锐， α 值越高。

即使是简单的一元和二元近似模型也产生了非零的 α 值

在英语到德语翻译的情况下，二元模型的 α 值为0.2，由于在这种情况下 $c=0$ ，因此实现了1.25X的推断速度改进

c 是一个成本系数，用于量化近似模型（ M_q ）相对于目标模型（ M_p ）的计算成本。当 $c=0$ 时，理论上表示近似模型 M_q 的计算成本忽略不计

M_p	M_q	SMPL	α
GPT-LIKE (97M)	UNIGRAM	T=0	0.03
GPT-LIKE (97M)	BIGRAM	T=0	0.05
GPT-LIKE (97M)	GPT-LIKE (6M)	T=0	0.88
GPT-LIKE (97M)	UNIGRAM	T=1	0.03
GPT-LIKE (97M)	BIGRAM	T=1	0.05
GPT-LIKE (97M)	GPT-LIKE (6M)	T=1	0.89
T5-XXL (ENDE)	UNIGRAM	T=0	0.08
T5-XXL (ENDE)	BIGRAM	T=0	0.20
T5-XXL (ENDE)	T5-SMALL	T=0	0.75
T5-XXL (ENDE)	T5-BASE	T=0	0.80
T5-XXL (ENDE)	T5-LARGE	T=0	0.82
T5-XXL (ENDE)	UNIGRAM	T=1	0.07
T5-XXL (ENDE)	BIGRAM	T=1	0.19
T5-XXL (ENDE)	T5-SMALL	T=1	0.62
T5-XXL (ENDE)	T5-BASE	T=1	0.68
T5-XXL (ENDE)	T5-LARGE	T=1	0.71
T5-XXL (CNNDM)	UNIGRAM	T=0	0.13
T5-XXL (CNNDM)	BIGRAM	T=0	0.23
T5-XXL (CNNDM)	T5-SMALL	T=0	0.65
T5-XXL (CNNDM)	T5-BASE	T=0	0.73
T5-XXL (CNNDM)	T5-LARGE	T=0	0.74
T5-XXL (CNNDM)	UNIGRAM	T=1	0.08
T5-XXL (CNNDM)	BIGRAM	T=1	0.16
T5-XXL (CNNDM)	T5-SMALL	T=1	0.53
T5-XXL (CNNDM)	T5-BASE	T=1	0.55
T5-XXL (CNNDM)	T5-LARGE	T=1	0.56
LAMDA (137B)	LAMDA (100M)	T=0	0.61
LAMDA (137B)	LAMDA (2B)	T=0	0.71
LAMDA (137B)	LAMDA (8B)	T=0	0.75
LAMDA (137B)	LAMDA (100M)	T=1	0.57
LAMDA (137B)	LAMDA (2B)	T=1	0.71
LAMDA (137B)	LAMDA (8B)	T=1	0.74

Summary (推测解码的过程 视频演示)



Summary

传统自回归：一个一个生成

Step 1: 输入 "我" → 输出 "爱"

Step 2: 输入 "我爱" → 输出 "中"

Step 3: 输入 "我爱中" → 输出 "国"

Step 4: 输入 "我爱中国" → 输出 EOS

大模型调用 4 次，生成 4 个 token

推测解码：猜 + 验证

草稿：小模型输入 "我"，快速猜出 "爱中国EOS" 四个 token

验证：大模型一次性验证这 4 个 draft token

假设实际应该是 "我爱中华"：

"爱" 命中，"中" 命中，"国" 没命中（应该是 "华"）

大模型调用 1 次，确认了 2 个 token + 修正 1 个

Thinking: 为什么有效？

只要统计上，草稿的平均命中 token 数 > 0 ，就有推理加速。并且推测解码不改变输出质量，即被拒绝的 token 会从修正分布中重新采样，数学上保证最终输出分布与原模型一致。

MEDUSA推理框架

MEDUSA推理框架

MEDUSA:

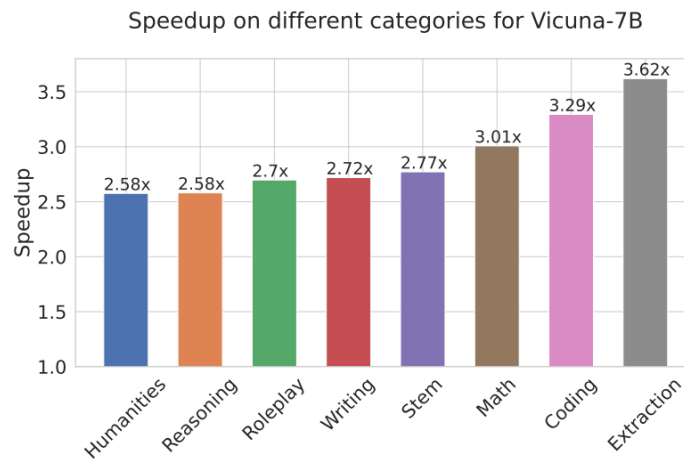
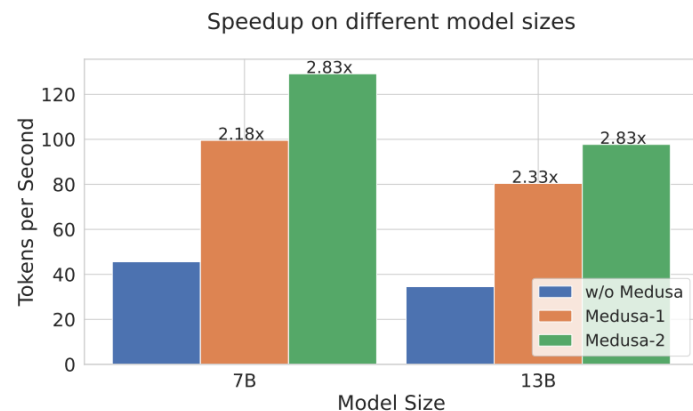
Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads <https://arxiv.org/abs/2401.10774>

- LLM通常会串行逐个生成token，成为Next-Token预测
- 使用Speculative Decoding 能巧妙的实现“并行解码”，但解码过程需要有小模型(Draft Model)参与，使得工程实现和部署并不够优雅。
- Medusa 提供了One Model 的并行解码方案，再现有模型上增加Multiple Decoding Heads 来做Next-Next-Token 预测，提高预测效率。Heads 类似美杜莎的形象



相较baseline, Medusa-2 加速2.83x

在Math/Coding/Extraction 推理任务中加速3x以上

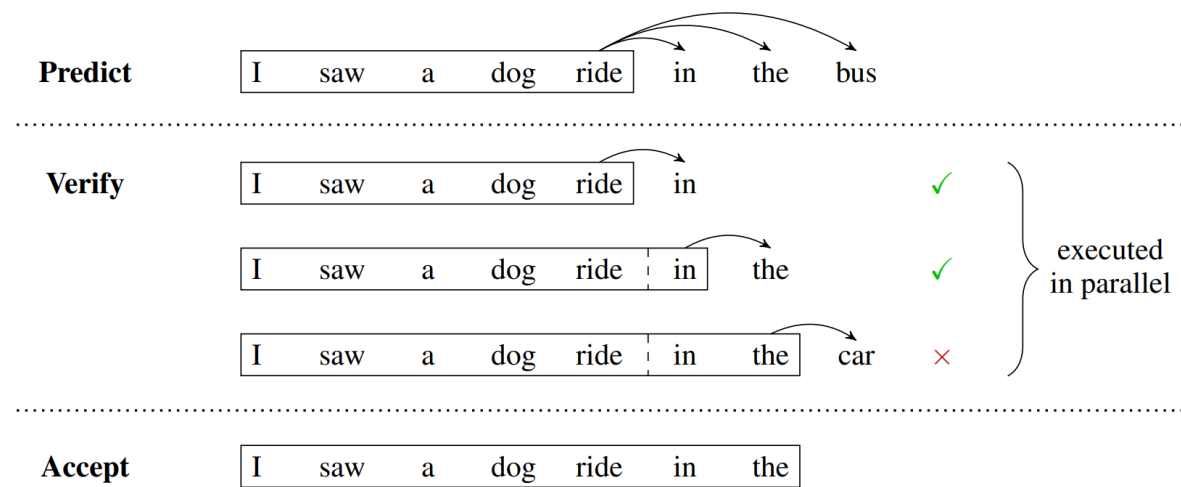


blockwise parallel decoding

blockwise parallel decoding（分组并行解码）：

https://proceedings.neurips.cc/paper_files/paper/2018/file/c4127b9194fe8562c64dc0f5bf2c93bc-Paper.pdf

- 论文提出的做法是Predict阶段，用scoring model生成k个token，然后在verify阶段使用proposal model验证各个token是否符合预期，这样verify阶段就能并行
- 在Accept阶段，第三个位置的最高概率预测是 car，它与独立预测的 bus不同。在Accept步骤中，只扩展到 in 和 the，然后再进行下一个k个独立的预测。



结合了scoring model和proposal model

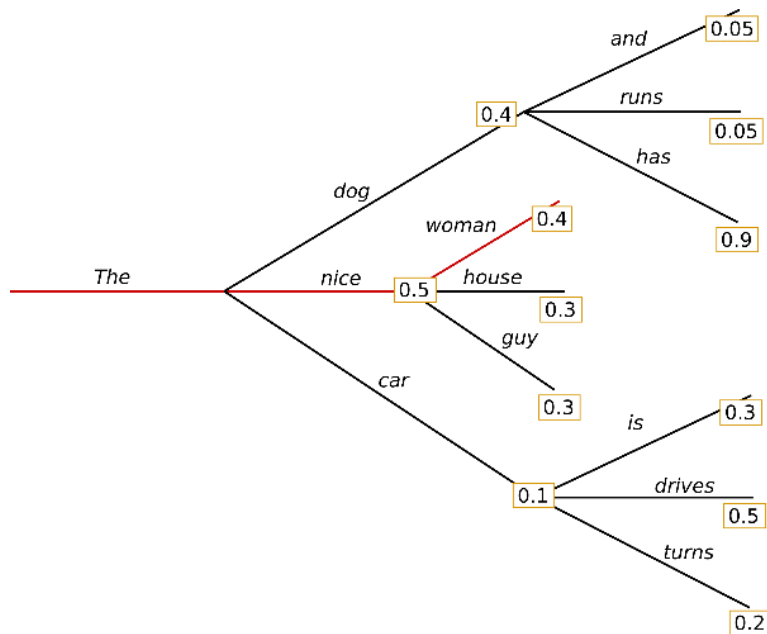
MEDUSA是Blockwise Parallel Decoding的继承与发展

三种解码方法生成文本

在Transformer中使用不用解码方式生成文本：

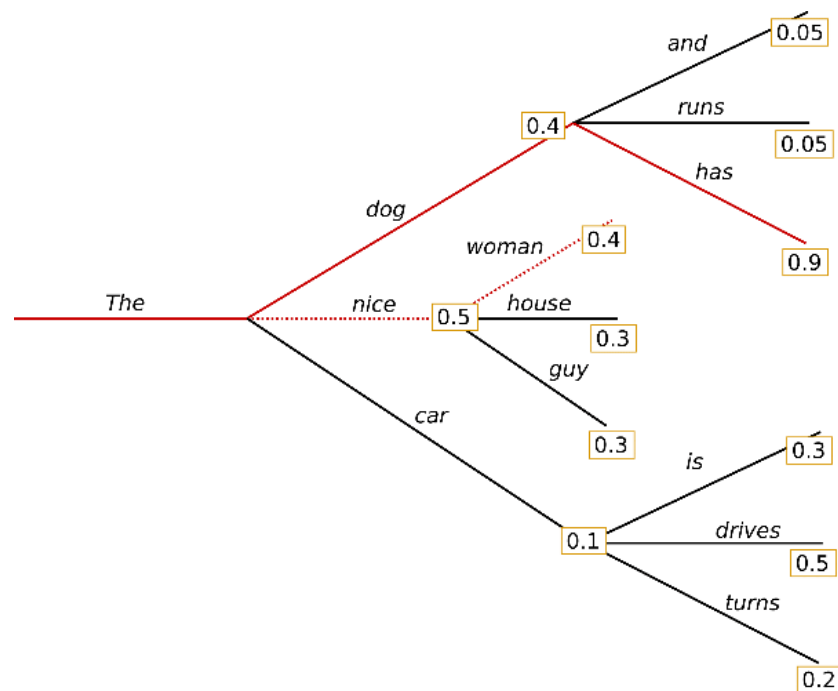
Greedy Search 贪心搜索

它选择概率最高的词作为其下一个词。优点是快，缺点是会错过隐藏在低概率词后面的高概率词，并且很容易生成重复的短语。



Beam search 束搜索

在每个step保持最可能的num_beams个生成候选束，并最终选择具有最高整体概率的候选束。会找到一个比Greedy Search概率更高的输出序列，但也不能保证找到联合概率最高的输出。



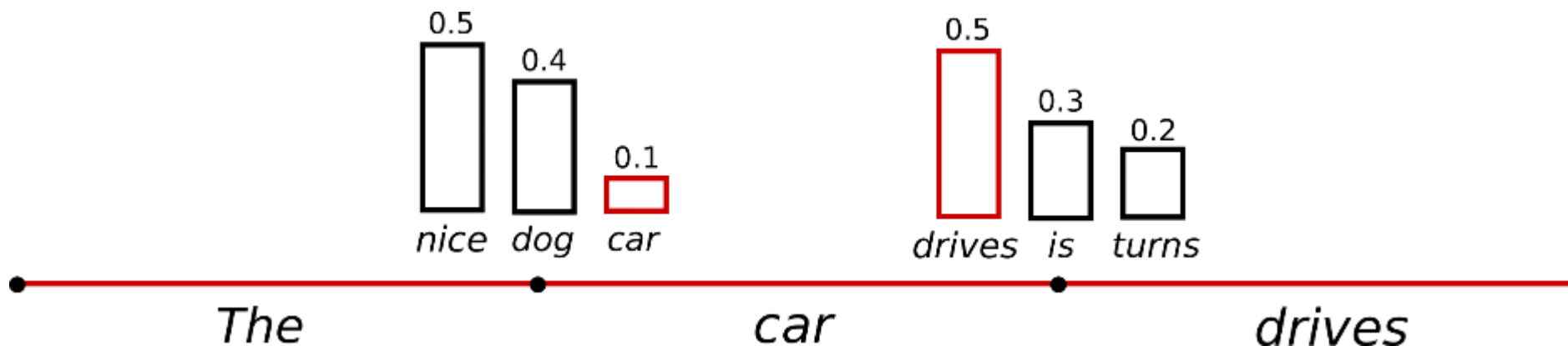
三种解码方法生成文本

Sampling 采样

以条件概率随机挑选下一个词。使用Sampling时文本生成本身不再是确定性的，小概率的token也有机会被生成。

在Top-K采样中，概率最大的K个词会被选出，然后这K个词的概率会被重新归一化，最后就在这重新被归一化概率后的K个词中采样。

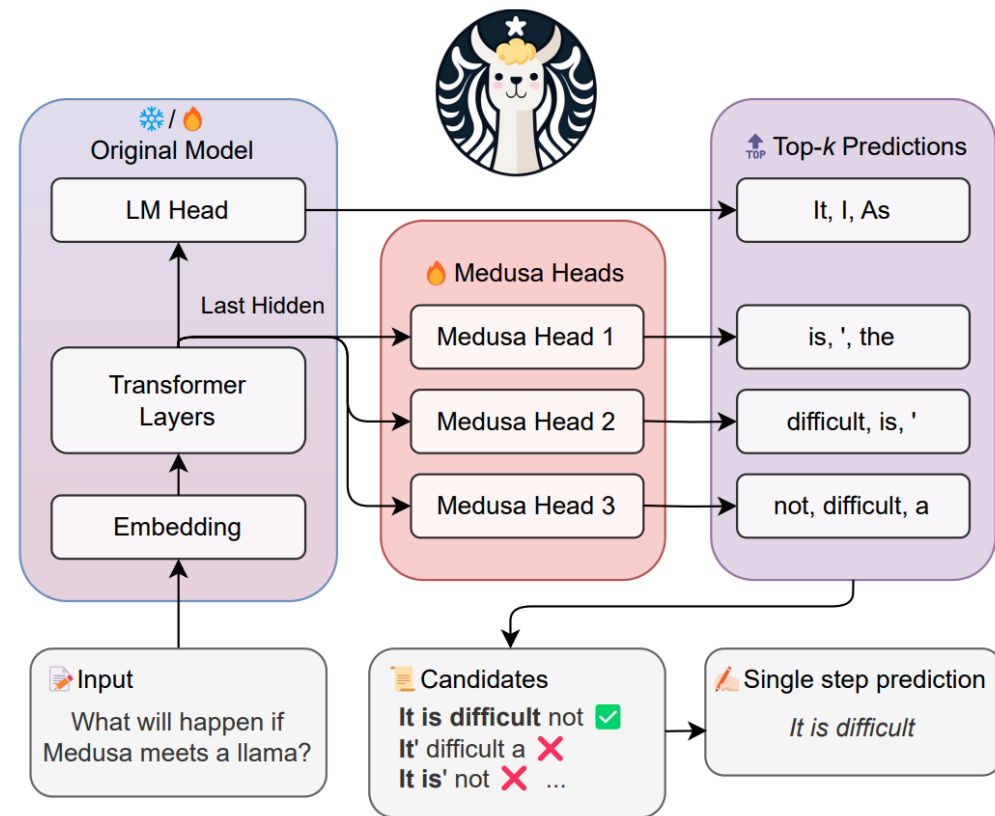
在Top-p中，采样不只是在最有可能的K个单词中进行，而是在累积概率超过概率p的最小单词集中进行。top-p和top-K采样可以结合使用，在开放式语言生成上产生比Greedy Search和Beam Search更为流畅的文本。



Medusa Heads

Medusa Heads:

- 在decoder后面增加多个LM_Head（也称project layer）。将LM_Head变成k个，可以预测后k个 tokens。
- Medusa在原始模型基础上，增加了3个额外的Head，可以并行预测出后4个token的候选。
- 训练这些Head简单。在训练阶段，原始模型保持frozen，只有Medusa Head经过微调，计算量比较少，微调比较快，能够在单个A100-80G GPU，一天时间内完成训练
- 如果我们每个Head都选择Top-1的预测做验证，就和Blockwise Parallel Decoding一样了。作者通过Vicuna模型测试发现，Top-1 head在预测next next token上只有60%准确率，这个效率其实很低。

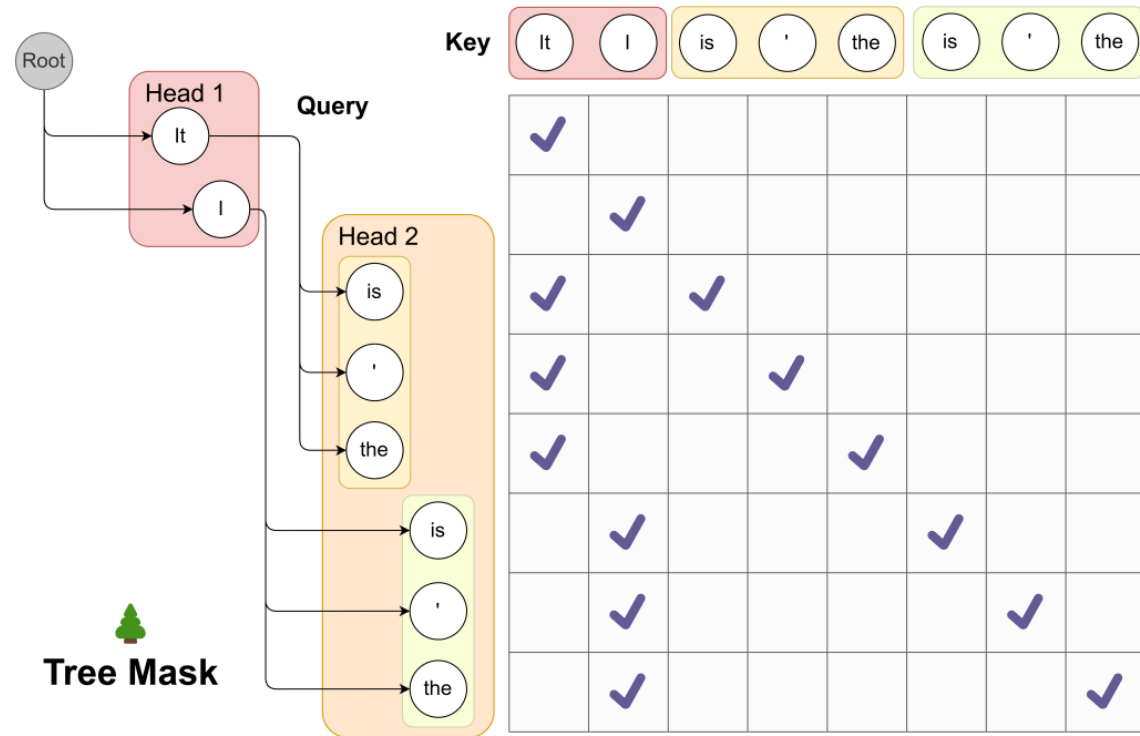


Medusa 在 LLM 的最后隐藏状态上引入了多个头部，使得可以并行预测多个后续标记。

Tree Attention

Tree Attention:

- 在Medusa中，Medusa Head基础版本解码采用greedy方式取Top-1 Token，实际上可以采用Top-k 得到多个候选token，构建出树状结构，LM_head 的输出作为根节点，树的深度自顶向下遍历称为解码路径 (论文称为 candidates path)。
- 右图使用第一个Head预测2个token，第二个head预测3个token。在这种情况下，第一个Head的任何预测都可以与第二个Head的任何预测配对，最终形成一个多层树结构。这棵树的每一层都对应于一个Medusa Head的预测。在这棵树内，Attention Mask需要新的设计，该Mask只限制对一个token的前面token的注意力。
- 在这个例子中，Attention Mask大小是 $(2+2*3) * (2+2*3) = 8*8$ 。



Tree Attention

在MEDUSA中，每个解码头可以并行预测多个后续的词。Tree Attention允许模型同时处理这些由不同解码头生成的候选词，显著提高处理速度。

Typical Acceptance (典型接受)

Typical Acceptance (典型接受) :

- MEDUSA框架中用来提高解码速度的策略。它是一种采样方法，用于从LLM生成的候选序列中选择合理的序列

=> 提高解码速度，同时保持生成文本的质量。

- 基于概率的接受标准

在传统的拒绝采样 (Rejection Sampling) 中，生成的候选序列需要与原始模型的输出分布完全一致才能被接受。而典型接受方法则放宽了这一要求，它接受那些在原始模型输出分布中具有合理概率的候选序列。这种策略允许一定程度的概率偏差，以换取更快的解码速度。

- 使用熵和阈值：

Typical Acceptance通过计算原始模型预测分布的熵 (Entropy) 来动态调整接受标准。如果一个候选序列的预测概率高于某个基于熵的阈值 (例如，熵的某个函数)，那么这个序列就被认为是“典型”的，即足够可能由原始模型生成，可以被接受。

- 温度参数：

在Typical Acceptance中，可以使用温度参数 (Temperature) 来控制接受标准的严格程度。

较高的温度值会使得更多的候选序列被接受，从而提高解码速度，但可能会牺牲生成质量。

Typical Acceptance (典型接受)

Medusa对Speculative Decoding的评价

- 随着温度增加，解码效率会急剧下降
- 尽管Draft model的分布对齐的很好，但输出的结构任有概率被拒接掉
- 在现实场景里，LLM通常被用来生成丰富的回答，用温度可以使得回答更有创造性，但是会导致输出分布不同，使得类似Speculative Sampling接受率低，Mudusa使用Typical Acceptance来提供接受率

Typical Decoding (典型解码)

- 一种基于信息的采样方法
- 直观理解是在LLM解码过程，不需要太能predictable的词，也不能有太surprising的词，这样就能保证我们能得到丰富且避免重复生成的词汇

predictable: 指的是条件概率下，生成的token可能性过大，一个例子是Donald 词后大概率会预测出Trump

surprising: 指的是sampling出不同寻常的词，导致生成的质量差，比如 今天星期 采样出 八

Typical Acceptance (典型接受)

Typical Decoding (典型解码) 算法流程

假设token于这个任务中，词汇表中的每个词预测的条件概率为

$$p_{\theta}(x|x_{>i})$$

每个词汇条件熵为计算公式如下:

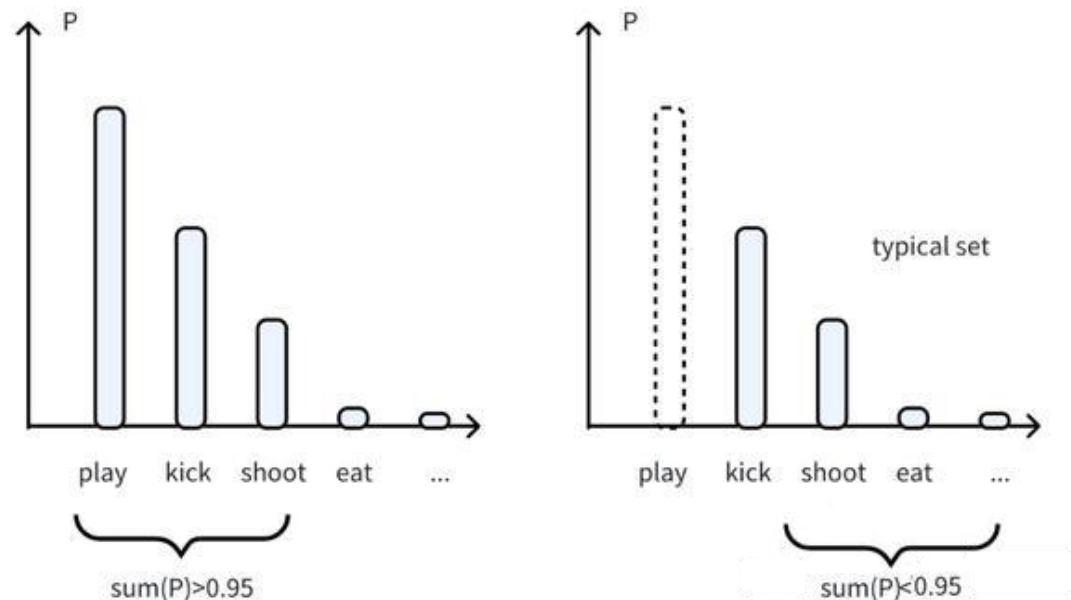
$$|h_{\theta,x_i} + \log p_{\theta}(x|x_{<i})|$$

基于条件熵(top-p按概率)按增序重排候选序列，由后往前数累加概率和<0.95，满足条件的token归入到Typical set，再进行sampling解码，这样就能避免才到过于Predictable的词汇了。

$$j = \arg \max_{j'} \sum_{i=1}^{j'} P_{\theta}(x^{(i)} | x_{<i}) \geq p$$

此时typical set为

$$\mathcal{A}_{x_{<i}} = \{x^1, \dots, x^j\}$$



Typical Acceptance理解

Thinking: Typical Acceptance（典型接受）要解决什么问题？

MEDUSA 用多个 Head 并行预测了多个候选 token，构成了一棵树。

现在问题来了：这棵树上这么多候选路径，哪些该接受、哪些该拒绝？

最简单的做法是用推测解码的标准验证（逐个比较概率），但 MEDUSA 提出了一种更灵活的策略——Typical Acceptance。

- Typical Acceptance的核心思想：

Typical Acceptance 的判断标准不是"概率最高才接受"，而是：接受那些"合理"的候选 token——既不太意外，也不太无聊。

它借鉴了 Typical Decoding（典型解码）的信息论思想：

类型	含义	例子
太 predictable	条件概率过高的词，信息量太低	"Donald" → "Trump"（几乎必然，没有新信息）
太 surprising	条件概率极低的词，不合常理	"Donald" → "蘑菇"（太离谱，质量差）
Typical（合理）	概率适中，既自然又有信息量	"Donald" → "announced"（合理且有内容）

Typical Acceptance理解

Thinking: 具体怎么判断?

关键靠两个东西: 熵 (Entropy) 和 阈值。

步骤:

- 计算原始模型在当前位置的预测分布的 熵 H (衡量不确定性)
- 对每个候选 token, 计算它的 信息量 (即 $-\log(p)$)
- 如果候选 token 的信息量 接近熵 H , 就认为它是"典型的", 予以接受

直觉理解:

- 熵 H 代表"模型此刻的平均不确定性"
- 一个 token 的信息量 $-\log(p)$ 如果和 H 差不多, 说明它"恰好符合预期的不确定程度"
- 太高 (太 surprising) 或太低 (太 predictable) 都会被过滤

Typical Acceptance理解

Thinking: 假设模型正在生成句子，当前要预测"The weather today is"后面的词：

模型预测分布：

- "sunny" → $p=0.35$, 信息量 $=-\log(0.35)=1.51$
- "good" → $p=0.25$, 信息量 $=-\log(0.25)=2.00$
- "nice" → $p=0.20$, 信息量 $=-\log(0.20)=2.32$
- "cold" → $p=0.10$, 信息量 $=-\log(0.10)=3.32$
- "banana" → $p=0.001$, 信息量 $=-\log(0.001)=9.97$

熵 $H \approx 2.1$ （当前分布的平均不确定性）

Typical Acceptance 的判断：

候选 token	信息量	与熵H的距离	判定
"sunny"	1.51	$ 1.51-2.1 =0.59$	接受（接近熵）
"good"	2.00	$ 2.00-2.1 =0.10$	接受（非常接近）
"nice"	2.32	$ 2.32-2.1 =0.22$	接受（接近熵）
"cold"	3.32	$ 3.32-2.1 =1.22$	拒绝（偏 surprising）
"banana"	9.97	$ 9.97-2.1 =7.87$	拒绝（太 surprising）

温度高 → 接受阈值更宽松 → 更多候选被接受 → 解码更快，但质量可能下降

温度低 → 接受阈值更严格 → 只接受最"典型"的 → 质量更高，但速度慢

Summary

MEDUSA-1在不牺牲生成质量的情况下，实现了2.2倍的速度提升

MEDUSA-2进一步提升了速度，达到了2.3到2.8倍的提升。

	Baseline	Direct Fine-tuning	MEDUSA-1	MEDUSA-2
Quality	6.17	5.925	6.23	6.18
Speedup	N/A	N/A	2.18	2.83

不同部署架构的Vicuna-7B的比较。质量是通过使用GPT-4作为评判在MT-Bench上评估模型得到的（数值越高越好）。

Model Name	Vicuna-7B	Zephyr-7B	Vicuna-13B	Vicuna-33B
Acc. rate	3.47	3.14	3.51	3.01
Overhead	1.22	1.18	1.23	1.27
Quality	6.18 (+0.01)	7.25 (-0.07)	6.43 (-0.14)	7.18 (+0.05)
$S_{\text{SpecDecoding}}$	1.47	-	1.56	1.60
S_{MEDUSA}	2.83	2.66	2.83	2.35

不同MEDUSA-2模型的比较。第一部分说明了MEDUSA-2的细节，包括加速度、开销和质量，这些是基于与原始模型相比在MTBench上的平均得分。

第二部分列出了SpecDecoding和MEDUSA的加速比(S)

Summary

MEDUSA 的动机

论文：《Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads》(2024)

=>核心思想：去掉独立的 Draft Model

传统推测解码需要一个独立的小模型做 Draft，工程上不够优雅（要维护两个模型）

MEDUSA 提供 One Model 的方案：在原始模型上增加多个解码头 (Multiple Decoding Heads)

不做 Next-Token 预测，而是做 Next-Next-Token 预测 -- 多个 Head 并行预测未来若干 token

前身：Blockwise Parallel Decoding (NeurIPS 2018)

Predict: 用 scoring model 生成 k 个 token → Verify: 验证是否符合预期（可并行）→ Accept: 只保留匹配部分。

MEDUSA 是这一方法的继承与发展。

Summary

Thinking: Medusa-1 和 Medusa-2 的区别是什么？

Medusa-1（冻结主干）

原理：保持原LLM参数完全冻结，仅训练新增的多个解码头（Medusa Heads）

生成质量：无损，与原始模型完全一致

加速比：约 2.2x

Medusa-2（联合训练）

原理：同时微调主干模型和Medusa Heads，使两者分布更对齐，提高Heads预测准确性

生成质量：几乎无损（通过特殊技巧保持模型能力）

加速比：约 2.8x

Summary

Medusa 的多头结构



在原始模型基础上增加 3 个额外 Head，一次前向传播并行预测后 4 个 token 的候选

训练什么：原始模型参数冻结，只微调 Medusa Head（参数量极少）

训练成本：单个 A100-80G GPU，一天内完成训练

Top-1 准确率：Vicuna 模型测试，next-next-token 的 Top-1 准确率约 60%

Summary

Thinking: 60% 的准确率够用吗?

如果每个 Head 只选 Top-1 做验证, 就退化为 Blockwise Parallel Decoding (每步最多接受 1 个额外 token)。60% 意味着经常猜错。所以 MEDUSA 不选 Top-1, 而是选 Top-k, 构建树状候选结构 -- 这就是 Tree Attention。

Medusa 的树状候选结构 (示例)



树的每一层对应一个 Medusa Head 的预测。树上的每条从根到叶的路径就是一个解码候选 (candidates path)。

Attention Mask 设计: 每个 token 只能看到自己和祖先节点 (不能看兄弟节点), 保证因果关系正确。所有候选路径一次前向传播并行验证。

EAGLE推理框架

EAGLE推理框架

Thinking: 为什么需要 EAGLE?

传统推测解码的三大痛点

- 独立 Draft 模型：需要维护两个模型（如 LLaMA-7B 给 LLaMA-70B 做草稿），部署复杂
- Token 级预测：离散 token 的不确定性高（如 "I" 后面可能是 "am"、"always"、"like"），预测难度大
- 静态结构：不管输入是简单算术还是复杂推理，草稿树结构固定，资源浪费

EAGLE推理框架

Speculative Decoding

- 独立 Draft 模型
- token $\rightarrow$ token
- 基于 token 预测 token
- 需要维护两个独立模型

MEDUSA

- 多头并行预测
- feature $\rightarrow$ token (独立)
- 每个 Head 独立使用特征预测 token
- Head 之间无交互

EAGLE

- 特征级自回归
- feature + token $\rightarrow$ feature $\rightarrow$ token
- 用特征序列 + token 序列预测下一个特征，再从特征采样 token

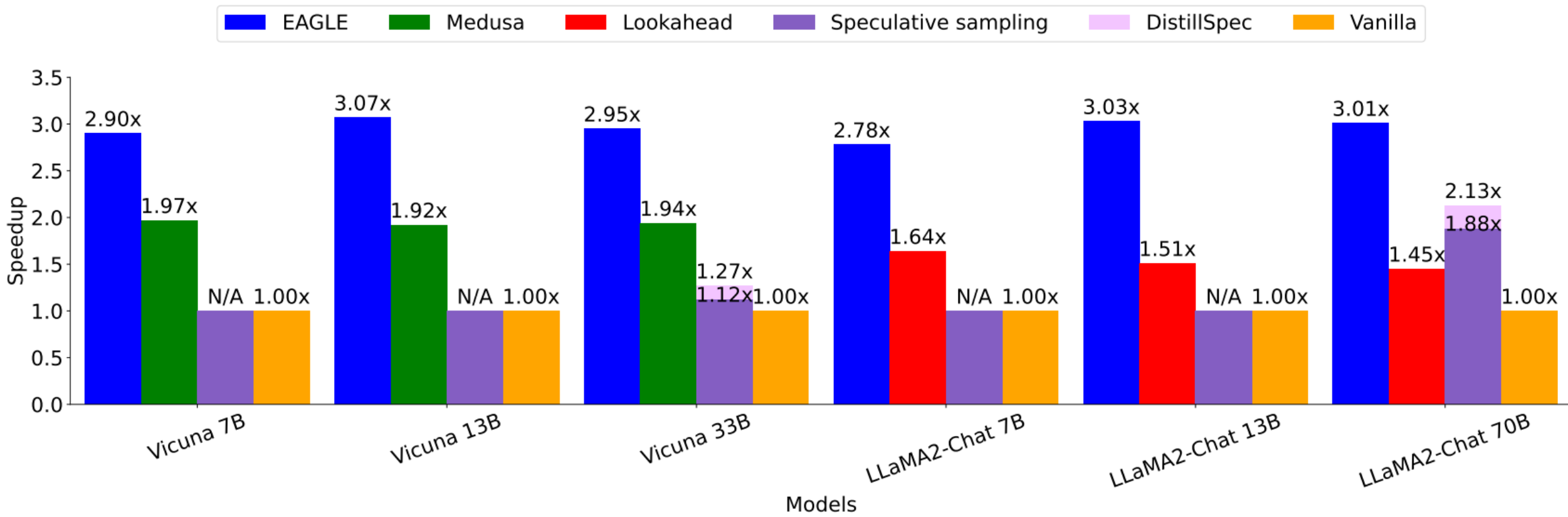
EAGLE推理框架

EAGLE推理框架 <https://github.com/SafeAILab/EAGLE>

EAGLE Speculative Sampling Requires Rethinking Feature Uncertainty, 2024 <https://arxiv.org/pdf/2401.15077>

EAGLE-2 Faster Inference of Language Models with Dynamic Draft Trees, 2024 <https://arxiv.org/pdf/2406.16858>

EAGLE-3: Scaling up Inference Acceleration of Large Language, 2025 <https://arxiv.org/pdf/2503.01840>



EAGLE推理框架

EAGLE推理框架 <https://github.com/SafeAILab/EAGLE>

- EAGLE is:
 - certified by the [third-party](#) evaluation as the **fastest** speculative method so far.
 - achieving **2x** speedup on [gpt-fast](#).
 - **3x** faster than vanilla decoding (13B).
 - **2x** faster than [Lookahead](#) (13B).
 - **1.6x** faster than [Medusa](#) (13B).
 - provably maintaining the consistency with vanilla decoding in the distribution of generated texts.
 - trainable (within 1-2 days) and testable on 8x RTX 3090 GPUs. So even the GPU poor can afford it.
 - combinable with other paralleled techniques such as vLLM, DeepSpeed, Mamba, FlashAttention, quantization, and hardware optimization.



EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees

Yuhui Li[♣] Fangyun Wei[‡] Chao Zhang[♣] Hongyang Zhang^{♣†}
♣Peking University ‡Microsoft Research ♣University of Waterloo †Vector Institute
hongyang.zhang@uwaterloo.ca
<https://github.com/SafeAILab/EAGLE>

Abstract

Inference with modern Large Language Models (LLMs) is expensive and time-consuming, and speculative sampling has proven to be an effective solution. Most speculative sampling methods such as EAGLE use a static draft tree, implicitly assuming that the acceptance rate of draft tokens depends only on their position. Interestingly, we found that the acceptance rate of draft tokens is also context-dependent. In this paper, building upon EAGLE, we propose EAGLE-2, which introduces a new technique of context-aware dynamic draft tree into drafting modeling. This improvement leverages the fact that the draft model of EAGLE is well-calibrated: the confidence scores from the draft model approximate acceptance rates with small errors. We conducted extensive evaluations on three series of LLMs and six tasks, with EAGLE-2 achieving speedup ratios 3.05x-4.26x, which is 20%-40% faster than EAGLE-1. EAGLE-2 also ensures that the distribution of the generated text remains unchanged, making it a **lossless** acceleration algorithm.

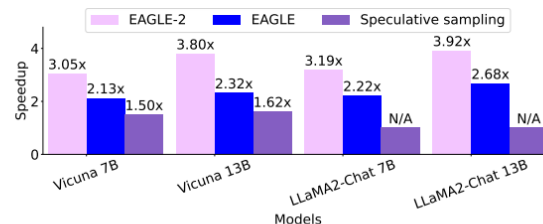


Figure 1: Speedup ratios of different methods at temperature=1. For speculative sampling, the Vicuna series uses Vicuna-68M as the draft model. LLaMA2-Chat lacks a suitable draft model, and is marked as N/A. Methods like Medusa relax acceptance conditions under non-greedy settings, which do not guarantee lossless acceleration. *In this paper, we only compare with speculative sampling based methods ensuring the output text distribution remains constant.* In Table 1, we present comparisons with additional methods, but this figure only showcases a subset, including the fastest among these methods, EAGLE.

Standard speculative sampling (Leviathan et al., 2023; Chen et al., 2023a) uses a chain-structured draft. To improve ac-

EAGLE推理框架

EAGLE (Extrapolation Algorithm for Greater Language-model Efficiency)

通过推测采样 (speculative sampling) 提高大型LLMs的推理效率。EAGLE的核心思想是在特征层面 (即模型的倒数第二层) 进行自回归处理, 而不是在token层面, 并通过将下一个时间步的token序列纳入draft model来解决预测下一个特征时的不确定性问题。

主要贡献:

- **特征层面的自回归更简单:** 在特征层面进行自回归比在token层面更直接, 因为特征序列比token序列更规则。
- **特征不确定性的处理:** EAGLE通过将下一个时间步的token序列作为输入, 有效地解决了特征序列预测中的不确定性问题。

- **高效的推理速度提升:** 在多种任务上的实验表明, EAGLE在保持生成文本分布不变的前提下, 能够显著提高推理速度。

对于LLaMA2-Chat 70B模型, EAGLE在对话、代码生成、数学推理和指令遵循任务上实现了2.7x-3.5x的延迟加速比。

- **低训练成本:** EAGLE的训练成本相对较低, 对于70B模型, 使用不到70k的对话数据在1-2天内即可完成训练。
- **通用性和可靠性:** EAGLE适用于任何自回归LLMs, 并且在实验中没有对原始LLM进行微调, 理论上保证了输出分布的保持。
- **与其他加速技术的兼容性:** EAGLE可以与其他加速技术 (如量化、编译等) 结合使用, 进一步降低LLM系统的运营成本。

EAGLE推理框架

Thinking: 为什么特征比 Token 更好预测?

比如: 输入 "The capital of France is", 预测下一个词

- Token 层面

可能是 "Paris"、"located"、"a"...

"Paris" 是词表中一个离散 ID, 与 "France" 无数学关联。

=> 离散、不连续、不确定性高

- 特征层面（倒数第二层隐藏状态）

"Paris" 对应的特征向量与 "France"、"capital" 的特征在语义空间中距离很近。

=> 连续、规则、时间平滑性强

草稿阶段

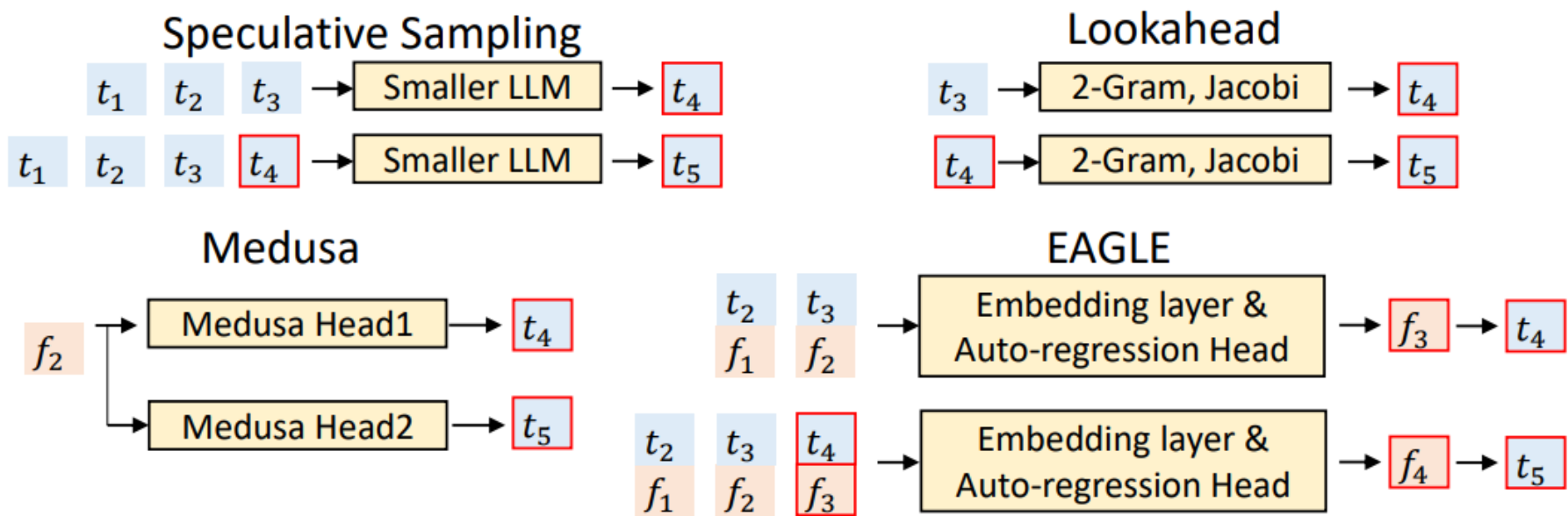
EAGLE与其他基于推测性采样的方法一样，包括草稿阶段和验证阶段。

EAGLE与其他方法的主要区别在于草稿阶段。下图展示了不同方法草稿阶段的示意图

推测性采样（Leviathan等人2023年）和Lookahead（Fu等人，2023年）基于token预测token。

Medusa（Cai等人，2023年）独立地使用目标LLM的特征f2预测t4和t5。

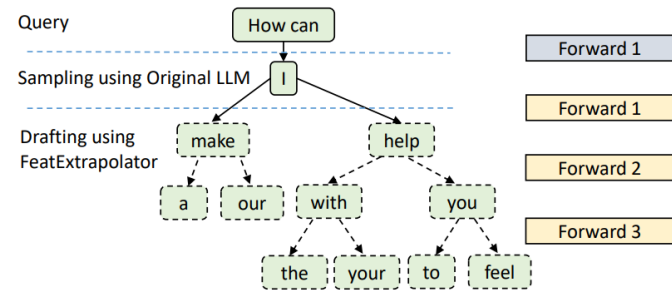
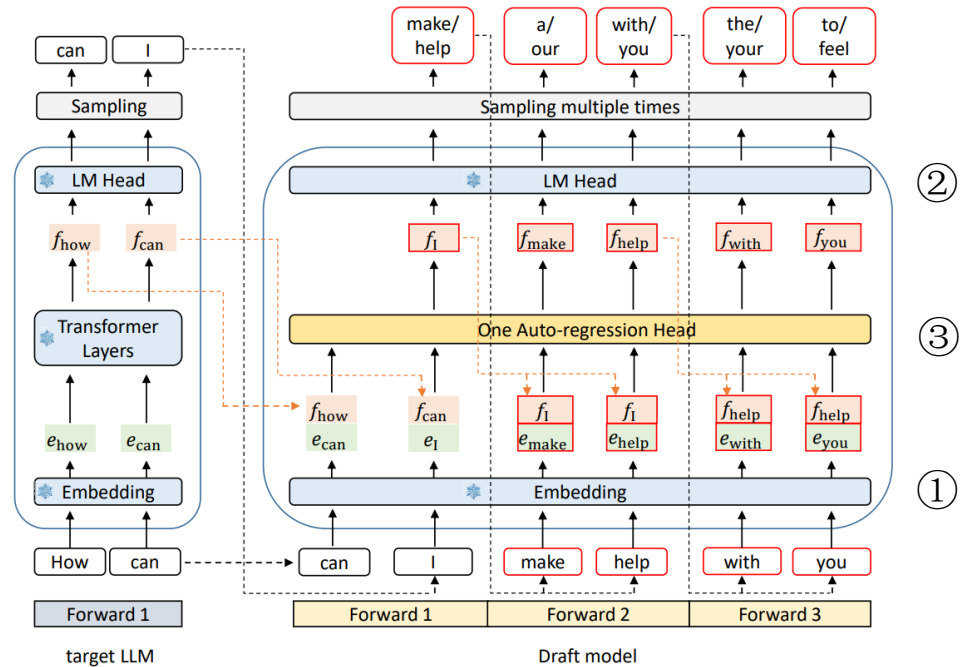
EAGLE使用特征序列（f1,f2）和提前一个时间步的token序列（t2,t3）预测f3，从p4=LM Head(f3) 中采样t4。随后，f3和t4被连接到输入序列中，以预测下一个特征f4并采样后续token t5。



EAGLE Draft Model

EAGLE Draft Model由三个模块组成：嵌入层、LM Head和自回归头部

- 嵌入层和LM Head使用目标LLM的参数，不需要额外训练。
- Draft model接受特征序列和提前一个时间步的token序列。然后将token序列转换为token嵌入序列，并将其连接起来形成一个融合序列。
- 自回归头部由一个FC层和一个解码器层组成。FC层将融合序列的维度降维，然后使用解码器层来预测下一个特征。LM Head根据特征计算分布，从中采样下一个token。
- 最后，预测的特征和采样的token被连接到输入中，以便于继续自回归过程。
- EAGLE使用Tree Attention创建一个树状草稿，通过m次前向传递生成深度为m且超过m个token的草稿树。右图所示，EAGLE仅用3次前向传递就草拟了一个10个token的树。



EAGLE pipeline, 在上半部分中，绿色块表示token嵌入，橙色块表示特征，红色框表示draft model的预测，带有雪花图标的蓝色模块表示目标LLM参数的使用，这些参数不受训练的约束。

EAGLE Draft Model训练

Draft Model训练:

预测下一个特征构成了一个回归任务，采用Smooth L1损失（右图）：

$$L_{reg} = \text{Smooth L1}(f_{i+1}, \text{Draft Model}(T_{2:i+1}, F_{1:i}))$$

预测特征是draft model的中间目标，最终目标是预测token以生成

token序列。因此，作者使用分类损失直接优化这个最终目标：

$$p_{i+2} = \text{Softmax}(\text{LM Head}(f_{i+1}))$$

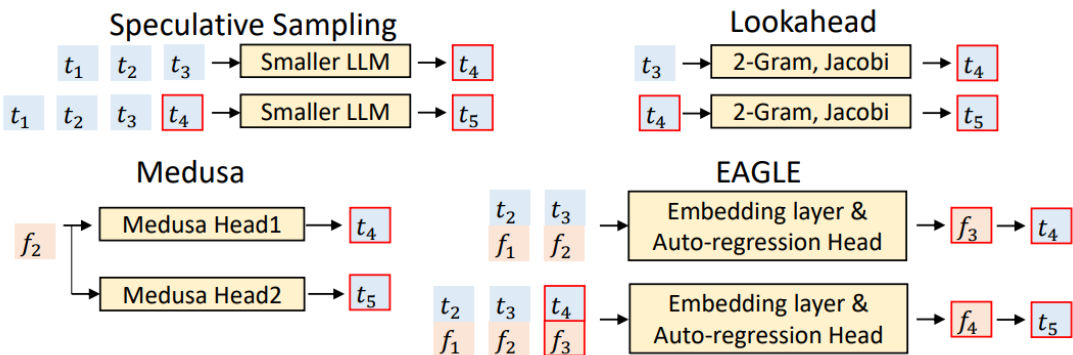
$$\hat{p}_{i+2} = \text{Softmax}(\text{LM Head}(\hat{f}_{i+1}))$$

$$L_{cls} = \text{Cross Entropy}(p_{i+2}, \hat{p}_{i+2})$$

通过整合回归损失和分类损失，使用组合损失函数 $L = L_{reg} + w_{cls}L_{cls}$

训练自回归头部。通常，分类损失在数值上比回归损失大一个数量级。因此，作者将 w_{cls} 设置为0.1。

EAGLE的自回归头部理想情况下是使用目标LLM自动生成的文本进行训练，但这种方法成本较高。幸运的是，EAGLE对训练数据的敏感性较低。作者不使用目标LLM生成的文本，而是使用一个固定的数据集，大幅减少了开销。



在草稿阶段，EAGLE自回归处理特征。特征的不准确性可能导致误差累积。为了缓解这个问题，作者在训练期间对目标LLM的特征添加随机噪声，这些噪声是从均匀分布 $U(-0.1, 0.1)$ 中采样的（Jain等人，2023年）。

验证阶段

验证阶段：

紧跟在草稿阶段之后。在草稿阶段，EAGLE生成了一个树状结构的草稿，其中包含多个候选token及其相应的概率分布。

验证阶段的目标是确保这些候选token符合目标LLMs的真实概率分布，从而保证生成文本的质量和多样性。

验证阶段的步骤：

- **使用树状注意力机制：**EAGLE利用树状注意力机制来处理草稿树中的每个节点。这种方法允许模型在单个前向传递中评估多个候选token，从而提高效率。
- **计算每个token的概率：**对于草稿树中的每个token，目标LLM计算其在目标分布中的真实概率。这是通过目标LLM的前向传递完成的，其中每个token的预测概率由模型的输出层给出。

- **递归应用推测性采样算法：**对于树状草稿中的每个节点，EAGLE递归地应用多轮推测性采样算法。检查每个候选token是否被接受，基于其在目标分布和草稿分布中的概率比率。

如果一个token被接受（即，其在目标分布中的概率与草稿分布中的概率之比大于一个随机阈值），则该token被保留。

如果token被拒绝，算法将调整目标分布（通过减去草稿分布中的概率，确保结果非负），然后再次尝试采样或选择新的token。

- **记录接受的token和特征：**

在验证过程中，被接受的token及其特征都被记录下来。这些信息将用于后续的草稿生成阶段，确保生成过程的连贯性和一致性。

验证阶段，通过模拟目标LLM的概率分布，保持输出分布的一致性，确保了生成质量与直接使用目标LLM的生成质量相当。

实验设置

- 模型和任务

使用Vicuna模型（7B、13B、33B）、LLaMA2-Chat模型（7B、13B、70B）以及Mixtral 8x7B Instruct模型进行实验。

评估了EAGLE在多种任务上的性能，包括多轮对话、代码生成、数学推理和指令遵循。

使用了MT-bench、HumanEval、GSM8K和Alpaca数据集进行评估。

- 训练数据

EAGLE使用固定数据集进行训练，避免了使用目标LLM生成训练数据时增加的开销。

消融研究表明，使用目标LLM的数据略微提高了性能，这表明EAGLE对训练数据的敏感性较低，并且证明了采用固定数据集方法来减少成本的合理性。

Training data	Speedup	τ
Fixed dataset	2.78x	3.62
Data generated by target LLM	2.88x	3.75

实验设置

• 指标

EAGLE的主要目标是减少推理延迟，而不是提高吞吐量。因此，使用以下指标来评估加速效果：

1) 墙钟加速比（Walltime Speedup Ratio）

相对于传统自回归解码的实际测试加速比。

2) 平均接受长度 τ

每个目标LLM的前向传递中平均接受的token数量。

3) 接受率 α

在草稿阶段生成的token中被接受的比例，这是衡量草稿模型准确性的一个重要指标。

	Model	τ	$0-\alpha$	$1-\alpha$	$2-\alpha$	$3-\alpha$	$4-\alpha$
T=0	Vicuna 7B	3.94	0.79	0.74	0.72	0.73	0.67
	Vicuna 13B	3.98	0.79	0.74	0.72	0.74	0.70
	Vicuna 33B	3.68	0.74	0.69	0.67	0.67	0.66
	LLaMA2-Chat 7B	3.62	0.76	0.69	0.67	0.68	0.68
	LLaMA2-Chat 13B	3.90	0.77	0.69	0.69	0.70	0.71
	LLaMA2-Chat 70B	3.81	0.75	0.69	0.65	0.64	0.64
T=1	Vicuna 7B	3.17	0.71	0.68	0.66	0.66	0.65
	Vicuna 13B	3.20	0.73	0.68	0.68	0.67	0.69
	Vicuna 33B	3.22	0.71	0.67	0.64	0.64	0.64
	LLaMA2-Chat 7B	3.30	0.71	0.66	0.66	0.66	0.64
	LLaMA2-Chat 13B	3.45	0.73	0.69	0.66	0.67	0.67
	LLaMA2-Chat 70B	3.46	0.73	0.67	0.64	0.66	0.65

$0-\alpha$ 代表的是在完全没有错误的理想情况下，草稿模型生成的token被目标LLM接受的概率。

$2-\alpha$ 、 $3-\alpha$ 、 $4-\alpha$ ：随着不准确特征数量的增加，误差累积效应更加明显，导致后续预测的准确度进一步下降。

实验设置

- 实验设置

固定了目标LLM，并在ShareGPT数据集上训练了EAGLE。使用了68,000个对话迭代，学习率设置为3e-5。采用了AdamW优化器，其beta值(β_1, β_2)设置为(0.9, 0.95)，并实现了0.5的梯度裁剪。

EAGLE的可训练参数对于7B、13B、33B和70B模型分别为0.24B、0.37B、0.56B和0.99B。

对于MoE模型Mixtral 8x7B，EAGLE的可训练参数为0.28B。

EAGLE的一个显著特点是训练成本低：70B模型的自回归头部可以在1-2天内A100 40G服务器上完成训练。

- 实验结果

EAGLE在不同模型和任务上都取得了显著的加速效果。

在MT-bench数据集上，对于LLaMA2-Chat 70B模型，EAGLE实现了2.7x-3.5x的延迟加速比，同时在保持生成文本分布不变的前提下，吞吐量提高了一倍。

		HumanEval		GSM8K		Alpaca	
		Speedup	a	Speedup	a	Speedup	a
T=0	V 7B	3.33x	4.29	3.01x	4.00	2.79x	3.86
	V13B	3.58x	4.39	3.08x	3.97	3.03x	3.95
	V 33B	3.67x	4.28	3.25x	3.94	2.97x	3.61
	LC 7B	3.17x	4.24	2.91x	3.82	2.78x	3.71
	LC 13B	3.76x	4.52	3.20x	4.03	3.01x	3.83
	LC 70B	3.52x	4.42	3.03x	3.93	2.97x	3.77
T=1	V 7B	2.39x	3.43	2.34x	3.29	2.21x	3.30
	V13B	2.65x	3.63	2.57x	3.60	2.45x	3.57
	V 33B	2.76x	3.62	2.77x	3.60	2.52x	3.32
	LC 7B	2.61x	3.79	2.40x	3.52	2.29x	3.33
	LC 13B	2.89x	3.78	2.82x	3.67	2.66x	3.55
	LC 70B	2.92x	3.76	2.74x	3.58	2.65x	3.47

实验设置

- 批量大小和吞吐量

在LLMs的推理中，内存是主要的瓶颈，这导致GPU的计算资源未能得到充分利用。推测性采样方法的目的在于更有效地利用GPU的计算资源。

随着批量大小的增加，加速比有所下降。使用Vicuna 7B作为目标LLM时，batch size=4时的加速比高于batch size=3，这是因为在EAGLE的验证阶段，目标LLM在单次前向传递中处理多个token，batch size=4时的处理速度比批量大小为3时更快。

相比之下，传统的自回归解码，目标LLM每次前向传递处理一个token，batch size=3和4时的速度几乎相同。

Batch size	1	2	3	4	Throughput
Vicuna 7B	2.90x	2.87x	2.65x	2.76x	1.97x
LLaMA2-Chat 70B	3.01x	2.81x	2.50x	2.40x	1.99x

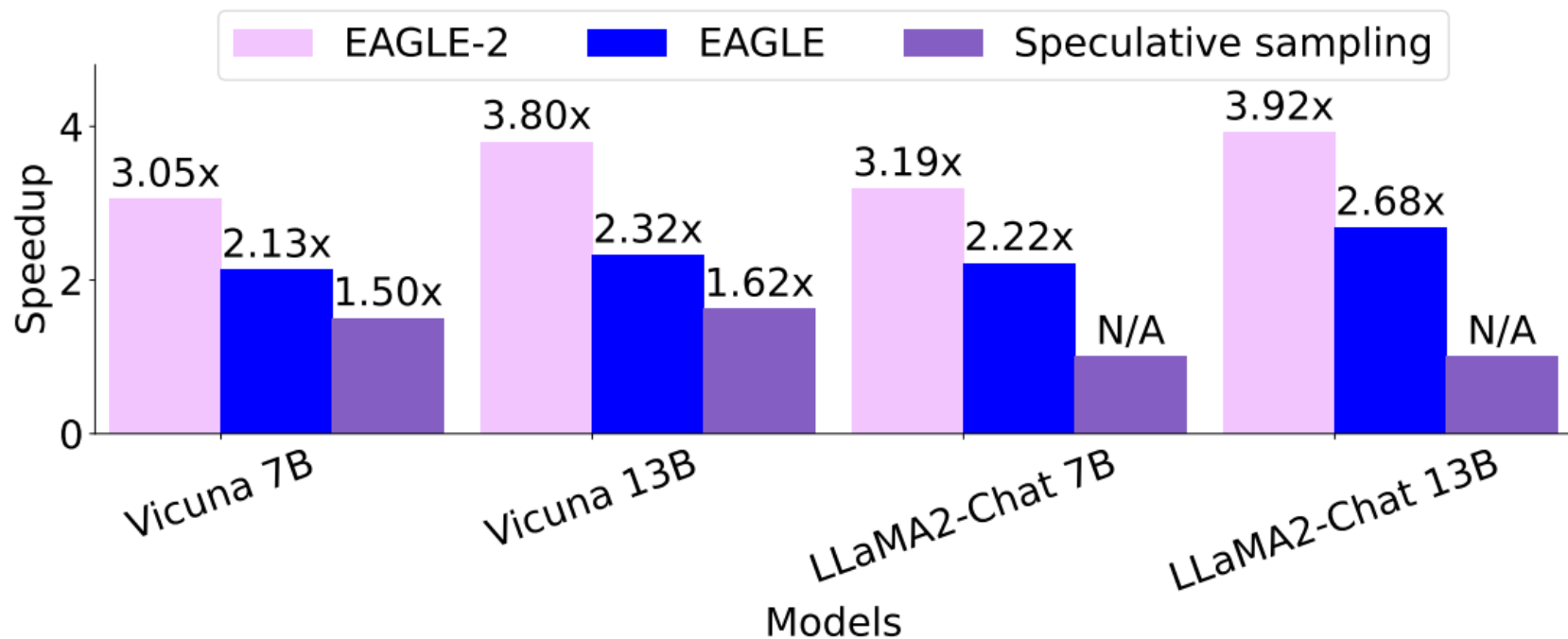
EAGLE2推理

EAGLE2推理

EAGLE-2: 在EAGLE的基础上进行了改进，引入了一种新的技术，即上下文感知的动态草稿树，用于草稿建模。

利用了EAGLE草稿模型的良好校准特性：草稿模型的置信度分数能够以小误差近似草稿token的接受率。

在多种任务和数据集上进行的广泛评估表明，EAGLE-2的加速比达到了3.05x-4.26x，比EAGLE-1快20%-40%。



EAGLE2推理

EAGLE2推理的主要贡献：

- **动态草稿树：** EAGLE-2根据草稿token的接受率动态调整草稿树结构，这与之前方法中使用的静态草稿树不同，能够更有效地生成和验证候选token。
- **保持文本分布不变：** EAGLE-2确保生成的文本分布与原始LLMs保持一致，实现了无损加速。
- **无需额外训练：** 与EAGLE相比，EAGLE-2不需要训练任何额外的模型，也不需要原始LLM进行微调或更新参数。

静态树 vs 动态树对比

针对: "10 + 2 =" 场景

- 静态树 (EAGLE-1)

Query: "10+2="

| - "1" (0.95) -- "2" (0.9)

| - "3" (0.05) -- "0" (0.04) <-- 浪费计算

- 动态树 (EAGLE-2)

Query: "10+2="

| - "1" (0.95) -- "2" (0.9) -- "." (0.85)

"3" 置信度低, 直接剪枝

计算资源集中在高概率分支

比 EAGLE-1 快 20%-40%, 加速比达到 3.05x-4.26x

动态草稿树 Dynamic Draft Trees

- 草稿树

在推测性采样中，草稿树是一种数据结构，用于存储和组织在生成过程中临时创建的候选token。

- 动态调整

与传统的静态草稿树（其结构在所有上下文中都固定不变）不同，动态草稿树根据当前的上下文信息和草稿token的预期接受率来调整其结构。

工作流程

- 扩展阶段（Expansion Phase）

EAGLE-2选择当前草稿树层中最有潜力的节点（即token），并使用草稿模型来生成下一层的候选token。选择哪些节点进行扩展是基于节点的“值”（一个结合了置信度和祖先节点置信度的度量）。

- 重排阶段（Reranking Phase）

扩展阶段的目标是加深草稿树，但选择的token可能在全局上不是最优的。因此，在重排阶段，EAGLE-2重新对所有草稿token进行排序，并选择具有最高值的前m个token。这确保了所选token仍然形成一个连接的树结构。

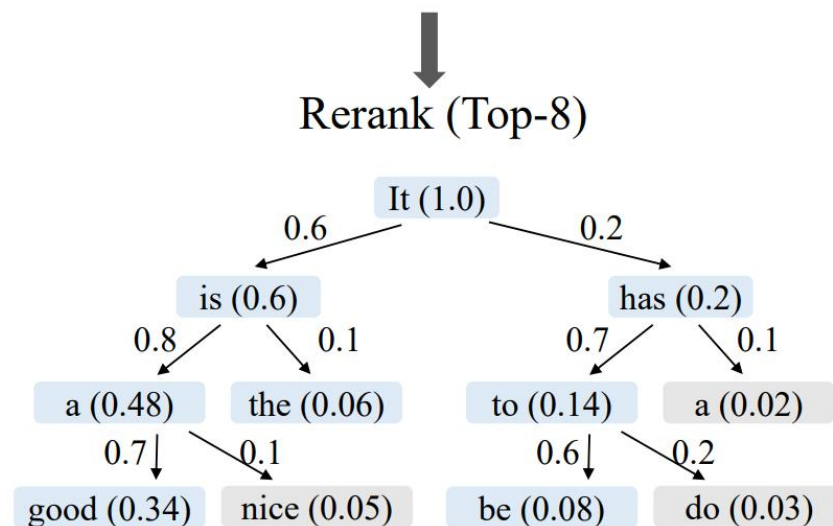
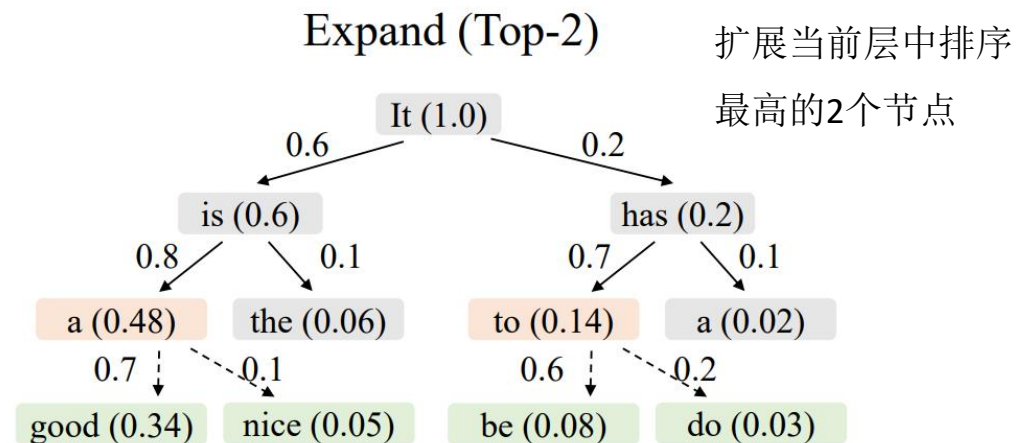
扩展阶段

- 扩展阶段

得益于Tree Attention, Draft Model可以同时输入当前层的所有token, 并在单次前向传递中计算下一个token的概率, 从而扩展当前层的所有token。然而, 一次性输入太多token会减慢Draft Model的前向传递速度, 而且Draft Tree每一层的token数量呈指数级增长。因此, 我们需要有选择性地扩展Draft Tree。

作者从当前层中选择具有最高全局接受概率的前k个token进行扩展。在推测性采样中, 如果拒绝了一个草稿token, 就会导致丢弃所有后续token; 一个token只有在其所有前缀都被接受时才最终被接受。Token t_i 的全局接受率是Draft Tree中从根节点到 t_i 节点路径上所有token接受率的乘积。我们将其定义为值 V_i :

$$V_i = \prod_{t_j \in \text{Path}(\text{root}, t_i)} p_j \approx \prod_{t_j \in \text{Path}(\text{root}, t_i)} c_j,$$



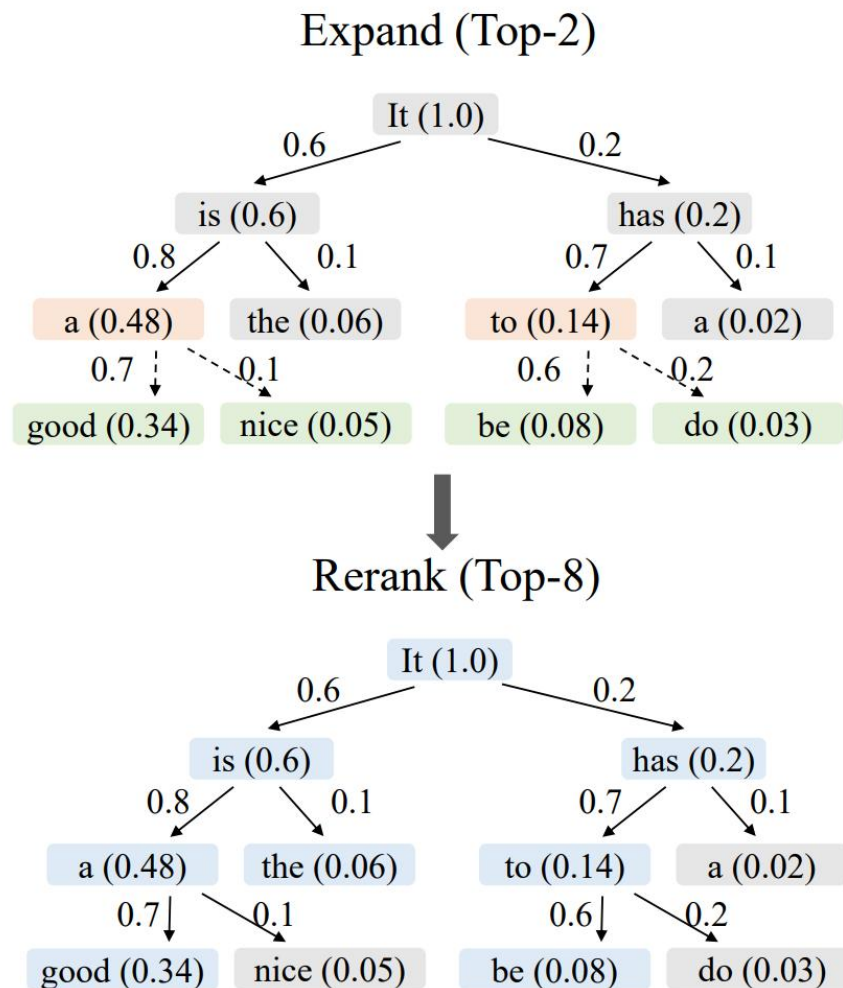
草稿树的扩展阶段

扩展阶段

其中 $\text{Path}(\text{root}, \text{ti})$ 代表草稿树中从根节点到 ti 节点的路径， pj 代表节点 tjtj 的接受率，而 cj 代表草稿模型中 tj 的置信度分数。

实验表明，置信度分数与接受率呈强正相关。我们利用这种关系来近似计算值。

从具有较高值的token开始的分支更有可能被接受。因此，我们选择上一次层中具有最高值的前k个节点作为输入到草稿模型，并根据输出来扩展草稿树。



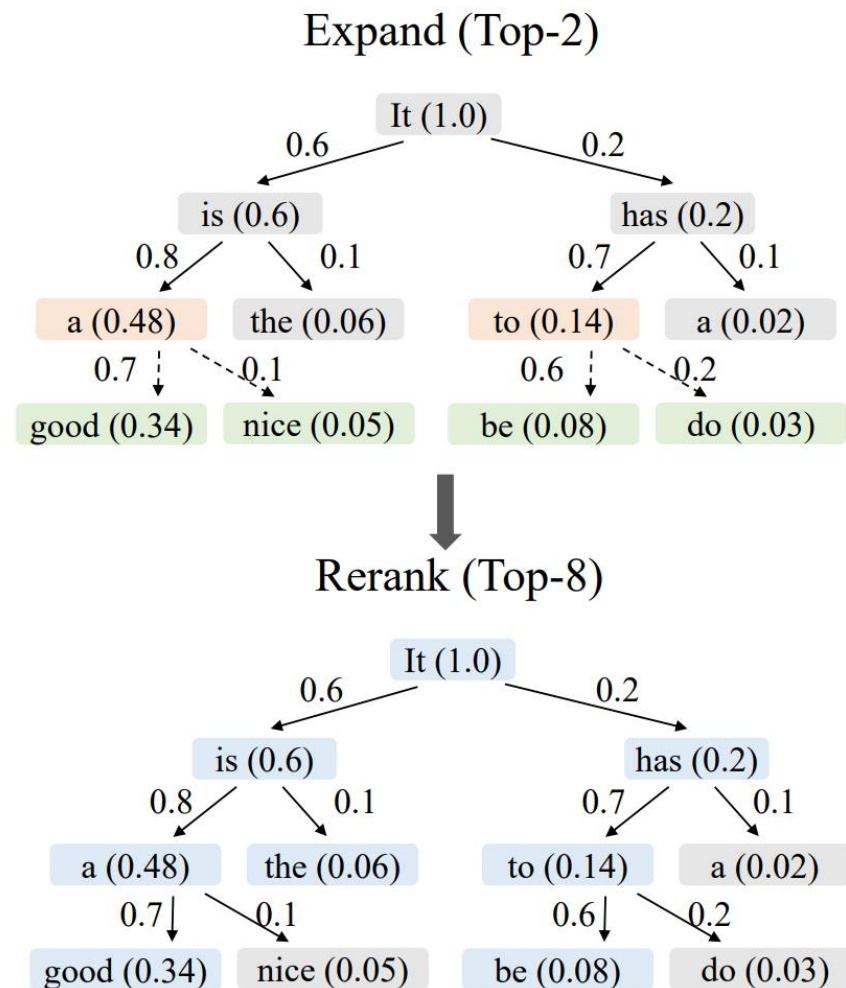
草稿树的扩展阶段

重排阶段

重排的目的：扩展阶段的目的是加深草稿树，但由于接受率在0到1之间变动，更深层的token的值会比较低。一些没有被扩展的较浅层节点可能具有比深层已扩展节点更高的值。**因此，扩展阶段选择的token并不直接作为草稿使用。这里，我们对所有草稿token重新进行排名，并选择值最高的前m个token。**

值的定义：一个节点的值总是小于或等于其父节点的值。对于具有相同值的节点，我们优先选择较浅的节点。这确保了重排后选出的前m个token仍然形成一棵连接的树。

一维序列：完成重排后，将选出的token展平成一维序列，作为原始LLM验证阶段的输入。



草稿树的扩展阶段

重排阶段

注意力掩码的调整：为了确保与原始自回归解码的一致性，还需要根据树结构调整注意力掩码。在原始自回归解码中，每个token可以看到所有前面的token，形成下三角的注意力矩阵。当使用草稿树时，来自不同分支的token不应该彼此可见。因此，必须根据树结构调整注意力掩码，以确保每个token只能看到其祖先节点。

重排的效果：通过重排操作，EAGLE-2能够提高平均接受长度和加速比，这是因为它确保了选择参与验证阶段的token是全局最优的。

		Attention mask							
		It	is	has	a	the	to	good	be
It		✓							
is		✓	✓						
has		✓		✓					
a		✓	✓		✓				
the		✓	✓			✓			
to		✓		✓			✓		
good		✓	✓		✓			✓	
be		✓		✓			✓		✓

注意力掩码

实验设置

模型：

在Vicuna 7B、13B、LLaMA2-Chat 7B、13B、70B和LLaMA3-Instruct 8B、70B模型上进行了实验。

任务：

六种生成任务上进行了全面的评估。对于多轮对话、代码生成、数学推理、指令遵循、摘要和问答任务

分别选择了MT-bench、HumanEval、GSM8K、Alpaca、CNN/Daily Mail和Natural Questions数据集。

遵循了LLMs社区中常用的零样本/少样本设置，即在所有任务中使用相同的草稿模型权重。

指标：

EAGLE-2既不对原始LLM进行微调，也不放宽接受条件，因此它是一种无损加速方法。因此不评估生成质量，而是使用以下指标来评估加速性能：

- **加速比：**相对于原始自回归解码的实际测试加速比。
- **平均接受长度 τ ：**每个起草-验证周期生成的token数量的平均值，这对应于从草稿中接受的token数量。平均接受长度的优点是它与硬件和运行时环境无关，但缺点是它不反映草稿模型的开销。

实验结果

在所有数据集和LLMs中，**EAGLE-2**实现了最高的加速比。大多数推测采样方法在代码生成任务（HumanEval）上表现出最高的加速比，这得益于代码中广泛使用的固定模板。

EAGLE在代码生成任务上实现了高达5倍的加速。

PLD在总结任务（CNN/DM）上使用Vicuna作为原始LLM时实现了最高的加速比，这得益于PLD基于检索的草稿生成和Vicuna执行总结时上下文的高重叠。

标准推测采样使用Vicuna-68M作为草稿模型，也实现了显著的加速比，但与其他方法相比，其训练开销要大得多。PLD和Lookahead不需要训练，而Medusa、Hydra、EAGLE和EAGLE-2使用SFT数据集训练草稿模型。Vicuna-68M同时使用预训练数据集和SFT数据集，其中预训练数据集比SFT数据集大很多。

		MT-bench		HumanEval		GSM8K		Alpaca		CNN/DM		Natural Ques.		Mean	
Model	Method	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ
Temperature=0															
V 13B	SpS	1.93x	2.27	2.23x	2.57	1.77x	2.01	1.76x	2.03	1.93x	2.33	1.66x	1.88	1.88x	2.18
	PLD	1.58x	1.63	1.85x	1.93	1.68x	1.73	1.16x	1.19	2.42x	2.50	1.14x	1.17	1.64x	1.69
	Medusa	2.07x	2.59	2.50x	2.78	2.23x	2.64	2.08x	2.45	1.71x	2.09	1.81x	2.10	2.07x	2.44
	Lookahead	1.65x	1.69	1.71x	1.75	1.81x	1.90	1.46x	1.51	1.46x	1.50	1.36x	1.39	1.58x	1.62
	Hydra	2.88x	3.65	3.28x	3.87	2.93x	3.66	2.86x	3.53	2.05x	2.81	2.11x	2.88	2.69x	3.40
	EAGLE	3.07x	3.98	3.58x	4.39	3.08x	3.97	3.03x	3.95	2.49x	3.52	2.42x	3.11	2.95x	3.82
	EAGLE-2	4.26x	4.83	4.96x	5.41	4.22x	4.79	4.25x	4.89	3.40x	4.21	3.13x	3.74	4.04x	4.65
L2 13B	PLD	1.42x	1.46	1.63x	1.70	1.41x	1.44	1.16x	1.20	1.42x	1.45	1.12x	1.15	1.36x	1.40
	Lookahead	1.58x	1.64	1.80x	1.85	1.65x	1.69	1.47x	1.50	1.46x	1.53	1.42x	1.45	1.56x	1.61
	EAGLE	3.03x	3.90	3.76x	4.52	3.20x	4.03	3.01x	3.83	2.70x	3.59	2.83x	3.47	3.09x	3.89
	EAGLE-2	4.21x	4.75	5.00x	5.52	4.31x	4.90	4.13x	4.61	3.45x	4.24	3.51x	4.04	4.10x	4.68
V 7B	SpS	1.82x	2.36	1.99x	2.61	1.71x	2.26	1.65x	2.21	1.81x	2.44	1.60x	2.16	1.76x	2.34
	PLD	1.61x	1.68	1.82x	1.87	1.82x	1.99	1.21x	1.31	2.53x	2.72	1.23x	1.44	1.70x	1.84
	Medusa	1.91x	2.52	2.02x	2.67	1.89x	2.59	1.79x	2.48	1.42x	2.02	1.51x	2.09	1.76x	2.40
	Lookahead	1.63x	1.69	1.72x	1.77	1.84x	1.99	1.38x	1.57	1.44x	1.53	1.45x	1.60	1.58x	1.69
	Hydra	2.69x	3.60	2.98x	3.79	2.73x	3.66	2.66x	3.58	2.01x	2.70	2.25x	2.86	2.55x	3.37
	EAGLE	2.90x	3.94	3.33x	4.29	3.01x	4.00	2.79x	3.89	2.33x	3.42	2.31x	3.21	2.78x	3.79
	EAGLE-2	3.62x	4.98	3.95x	5.33	3.63x	4.97	3.46x	4.86	2.94x	4.12	2.76x	3.82	3.39x	4.68
L2 7B	PLD	1.38x	1.43	1.52x	1.59	1.32x	1.37	1.15x	1.19	1.48x	1.52	1.15x	1.20	1.33x	1.38
	Lookahead	1.61x	1.66	1.72x	1.77	1.58x	1.65	1.49x	1.52	1.49x	1.54	1.48x	1.53	1.56x	1.61
	EAGLE	2.78x	3.62	3.17x	4.24	2.91x	3.82	2.78x	3.71	2.43x	3.41	2.61x	3.44	2.78x	3.71
	EAGLE-2	3.43x	4.70	4.03x	5.39	3.52x	4.77	3.45x	4.66	3.01x	4.12	3.15x	4.19	3.43x	4.64
Temperature=1															
V 13B	SpS	1.62x	1.84	1.72x	1.97	1.46x	1.73	1.52x	1.78	1.66x	1.89	1.43x	1.70	1.55x	1.82
	EAGLE	2.32x	3.20	2.65x	3.63	2.57x	3.60	2.45x	3.57	2.23x	3.26	2.14x	3.06	2.39x	3.39
	EAGLE-2	3.80x	4.40	4.22x	4.89	3.77x	4.41	3.78x	4.37	3.25x	3.97	3.07x	3.54	3.65x	4.26
L2 13B	EAGLE	2.68x	3.45	2.89x	3.78	2.82x	3.67	2.66x	3.55	2.41x	3.39	2.37x	3.31	2.64x	3.53
	EAGLE-2	3.92x	4.51	4.58x	5.29	4.21x	4.80	3.85x	4.48	3.31x	4.08	3.43x	3.89	3.88x	4.51
V 7B	SpS	1.50x	1.87	1.55x	1.95	1.53x	1.82	1.56x	1.85	1.63x	1.91	1.33x	1.72	1.52x	1.85
	EAGLE	2.13x	3.17	2.39x	3.43	2.34x	3.29	2.21x	3.30	2.08x	3.12	1.95x	2.86	2.18x	3.20
	EAGLE-2	3.05x	4.28	3.33x	4.65	3.07x	4.49	3.08x	4.43	2.63x	3.76	2.48x	3.56	2.94x	4.20
L2 7B	EAGLE	2.22x	3.30	2.61x	3.79	2.40x	3.52	2.29x	3.33	2.19x	3.15	2.22x	3.12	2.32x	3.37
	EAGLE-2	3.19x	4.41	3.67x	5.06	3.35x	4.62	3.20x	4.48	2.73x	3.85	2.81x	4.01	3.15x	4.41

EAGLE3推理

EAGLE-1和2 的隐藏瓶颈

EAGLE-1 和 2的瓶颈：数据量增加，加速比却不增长

现象：将训练数据从 ShareGPT 68k 扩展到 8 倍，EAGLE-1/2 的加速比提升非常有限（趋于饱和）。

根本原因有两个：

- 特征预测约束：EAGLE 强制 Draft Model 输出必须匹配目标模型的顶层特征，限制了模型的表达能力
- 训练-推理不一致：

训练时：输入是目标模型的真实特征 $f_1, f_2 \dots$ （标准答案）

推理时：输入包含 Draft Model 自己生成的特征（存在误差）

随着生成步数增加，误差累积，后续接受率骤降

EAGLE-1和2 的隐藏瓶颈

误差累积的影响：

EAGLE-1/2

- 第 1 步接受率 (α_0) 约 0.8
- 第 5 步接受率 (α_4) 降至 0.6
- 误差越积越大，后续猜测越不准

EAGLE-3

- 各步接受率均保持在 ~ 0.8
- 通过训练机制改进，有效抑制误差累积

EAGLE3推理

创新: Training-time Test (训练时测试)

核心思想: 在训练过程中模拟推理时的多步生成, 让 Draft Model 适应自己的预测结果, 而不是只见过"标准答案"。

传统 EAGLE 训练 (单步)

每次只做一步预测

输入: f_1, f_2 (真实特征)

输出: 预测 f_3 (拟合真实特征)

问题: 训练时只见过真实特征

推理时遇到自己的错误输出就懵了

EAGLE-3 训练 (多步模拟)

模拟推理时的多步生成

Step1: 输入 f_1, f_2 (真实)

生成 a_1 (Draft 的输出)

Step2: 输入 $f_1, f_2 + a_1$ (混合输入)

生成 a_2 # 模拟误差累积

Step3: 输入 $f_1, f_2 + a_1, a_2$

...

EAGLE-1 的训练像"看着标准答案做练习题" -- 练习时都对, 上考场 (推理) 遇到自己写错的中间步骤就不会了。

EAGLE-3 的训练像"闭卷考试 + 自我修正" -- 训练时就适应了自己可能犯错的情况, 上考场时更从容。

Summary

版本	核心突破	解决的关键问题	加速比
EAGLE-1	特征级预测 + Shifted-Token	Token 预测不确定性高	~3.0x
EAGLE-2	动态草稿树 (Context-aware)	静态树结构浪费计算资源	~4.2x
EAGLE-3	Training-time Test + 多层特征融合	特征约束限制 Scaling + 训练推理不一致	~6.5x

大 Batch 场景下的实际价值（SGLang 测试，Batch=64）

- EAGLE-1/2: 吞吐量降至 0.99x（几乎无加速，因为大 Batch 已是 Compute-Bound）
- EAGLE-3: 吞吐量仍保持 1.38x（在高并发生产环境中价值巨大）

Summary

大 Batch 场景下的实际价值（SGLang 测试，Batch=64）

- EAGLE-1/2: 吞吐量降至 0.99x（几乎无加速，因为大 Batch 已是 Compute-Bound）
- EAGLE-3: 吞吐量仍保持 1.38x（在高并发生产环境中价值巨大）

EAGLE-3 对高并发的意义：

EAGLE-1/2 在 Batch=64 时确实失效（0.99x），但 EAGLE-3 通过更高的接受率和 Scaling Law 仍能保持 1.38x 吞吐提升。

=> 真实的高并发生产环境中，EAGLE-3 是目前唯一能同时兼顾 大 Batch 高吞吐 和 推测解码加速 的方案。

Summary

Thinking: Batch Size 大了，会减弱推测解码效果么？

会的，类似拼车效果减弱

把 GPU 在 Decode 阶段的一次前向 想成 一辆车跑一趟：

车上的人 $\approx$ 当前 batch 里并行处理的请求 / 工作量

拼车 $\approx$ 在本来没坐满的车上多拉几个人，让这一趟路更划算

推测解码，可以理解为：除了正常解码，再让草稿模型多算一些 猜的 token，再用大模型验证。

- Batch 小 $\Rightarrow$ Decode 偏 Memory-Bound（算力没吃满）

车上只有 1 个人：再拼上推测解码带来的额外计算，利用率上去，加速明显。

- Batch 已经很大 $\Rightarrow$ GPU 已接近 Compute-Bound（算力快饱和）

车上已经挤了 30 人：再硬塞推测解码 这层草稿计算 $\Rightarrow$ 边际收益很小，草稿开销还可能 拖后腿

Summary

Thinking: 为什么EAGLE-1/2 不行（针对大 Batch）？

训练时输入的是目标模型的真实特征（标准答案），推理时输入包含自己上一步生成的特征（有误差）

随着推测步数增加，误差累积，第 5 步接受率从 ~ 0.8 降到 ~ 0.6

而且数据量扩 8 倍，加速比几乎不涨（Scaling 饱和）

所以在 Batch=64 时，EAGLE-1/2 的草稿质量本就一般，再加上大 Batch 下额外计算开销更贵，两者对冲后吞吐量 0.99x——约等于没加速。

EAGLE-1/2 的训练像 看着标准答案做题，推理时遇到自己的错误中间结果就懵了。

EAGLE-3 改成了闭卷考试 + 自我修正；

效果：各步接受率均保持在 ~ 0.8 ，不再像 EAGLE-1/2 那样从 0.8 衰减到 0.6。误差累积被有效抑制。

推测解码与高并发架构 的融合

推测解码在 vLLM 中的集成

推测解码与 vLLM 的 Continuous Batching 结合时有三个冲突点：

- 草稿阶段需要 batch=1，Continuous Batching 需要动态 batch

解决方案：草稿阶段作为独立 micro-batch，验证阶段合并入主 batch

- MEDUSA Tree Attention 需要特殊 Mask

解决方案：vLLM 支持 Custom Attention Backend

- 显存碎片（草稿模型 + KV Cache）

解决方案：PagedAttention 统一管理草稿和目标的 KV Cache

EAGLE 在 vLLM 中的工作流程

EAGLE + vLLM 单次 Iteration 流程

- Step1, 选择候选请求

选择 batch 中长度最长或历史接受率高的请求

- Step2, 草稿阶段 (Draft Stage)

使用 EAGLE draft model (0.24B 参数, 极轻量)

树状采样 3-5 步, 生成候选 token 树

- Step3, 验证阶段 (Verify Stage)

目标模型通过 Tree Attention 并行验证所有 draft tokens

- Step4, 更新 Block Table

保留被接受 tokens 的 blocks, 释放被拒绝的 blocks

vLLM + EAGLE 的 Iteration 流程

vLLM + EAGLE 的 Iteration 流程

1. 选择适合推测的请求

draft_requests = select_for_speculation(current_batch)

2. 草稿阶段 - 轻量级 EAGLE 模型快速预测

for step in range(max_draft_tokens): # 通常 3-5 步

 draft_features = eagle_model.forward(

 target_features, previous_tokens

)

 draft_tokens = sample(draft_features) # 树状采样

3. 验证阶段 - 目标模型并行验证所有候选

verified_tokens, accepted_length = tree_verify(

 draft_tokens, target_model_logits

)

4. 更新 PagedAttention Block Table

update_block_table(request, accepted_length)

vLLM 性能监控与调优

关键性能指标 (KPIs)

LLM 推理中的关键指标

延迟类指标

TTFT (Time To First Token)

从请求发送到第一个 token 输出的时间

受 Prefill 阶段影响

TPOT (Time Per Output Token)

相邻输出 token 间隔

受 Decode 阶段 batch size 影响

TTFB (Time To First Byte)

首字节网络返回时间 (含队列等待)

吞吐类指标

Throughput

tokens/second (总生成量 / 总时间)

Goodput

满足延迟约束的请求数/秒

如 TTFT < 1s, TPOT < 100ms

GPU 利用率

Compute Utilization (非显存占用)

推测解码指标

接受率 α

被接受 draft tokens / 总 draft tokens

>0.6 良好 <0.4 需调整

有效批量

Real BS x (1 + α x draft_len)

反映推测解码的实际加速效果

开销比

Draft / Target 推理时间比

EAGLE 5-10% MEDUSA 8-12%

接受率就像 猜题命中率：草稿模型猜了 5 个 token，目标模型验证后接受了 3 个，那么 $\alpha = 3/5 = 0.6$ 。

接受率越高，推测解码加速越明显；如果太低（猜不准），反而多花了草稿模型的计算开销却没省多少事。

vLLM 核心优化参数

并行与分布式

参数	说明	推荐值
--tensor-parallel-size	张量并行 (TP) 切分 GPU 数	根据 GPU 数设定 (2, 4, 8)
--pipeline-parallel-size	流水线并行 (PP) 层级数	大模型跨节点时使用
--data-parallel-size	数据并行 (DP) 实例数	高并发场景

内存与 KV Cache

参数	说明	推荐值
--max-model-len	最大序列长度	按实际需求, 避免浪费显存
--max-num-batched-tokens	最大批处理 token 数	受显存限制, 越大吞吐越高
--max-num-seqs	最大并发序列数	根据显存和序列长度调整
--kv-cache-dtype	KV Cache 数据类型	fp8 可省显存, 损失 <1%
--enable-chunked-prefill	启用分块预填充	强烈建议开启
--enable-prefix-caching	前缀缓存	多轮对话场景必开
--gpu-memory-utilization	GPU 显存利用率上限	0.90-0.95

vLLM 核心优化参数

量化与压缩

参数	说明
--quantization	量化方式: awq, gptq, fp8, int8 等
--load-format	加载格式: auto, safetensors (更快)
--dtype	计算精度: float16, bfloat16, float32

推测解码 (EAGLE/MEDUSA)

参数	说明
--speculative-model	草稿模型路径 (HuggingFace 格式)
--num-speculative-tokens	推测 token 数 (建议 3-5)
--speculative-draft-tensor-parallel-size	草稿模型的 TP (通常 1, 因为草稿模型小)
--speculative-max-model-len	草稿模型最大长度

vLLM 核心优化参数

调度与抢占

参数	说明
--scheduling-policy	调度策略：fcfs（先来先服务）或 priority
--preemption-mode	抢占模式：swap（换出到 CPU）或 recompute（重算）

监控工具栈

1. vLLM 内置 Metrics（Prometheus 格式）

通过 --enable-metrics 暴露, curl http://localhost:8000/metrics 查看

vllm:prompt_tokens_total # 输入 token 累计

vllm:generation_tokens_total # 生成 token 累计

vllm:time_to_first_token_seconds # TTFT 分布 (P50/P99)

vllm:time_per_output_token_seconds # TPOT 分布

vllm:gpu_kv_cache_usage_percent # KV Cache 利用率

vllm:num_requests_running # 当前运行中的请求数

vllm:num_requests_waiting # 队列等待中的请求数

vllm:spec_decode_draft_acceptance_rate # 推测解码接受率 (v0.8.5+)

监控工具栈

2. NVIDIA Nsight Systems（深度 Kernel 分析）

分析 kernel 执行时间，找出 Attention 是否是瓶颈

```
nsys profile -o vllm_profile python -m vllm.entrypoints.openai.api_server
```

3. 自定义 Grafana Dashboard 关键 Panel

实时监控

- 当前 batch 中的请求数
- GPU 计算利用率
- 队列等待请求数

延迟分布

- TTFT P99 (按序列长度分桶)
- TPOT P95

吞吐趋势

- 每分钟生成 token 数
- 请求完成率

推测解码

- 接受率趋势
- 草稿长度分布
- Draft Model 负载

场景化调优策略

目标 A: 最高吞吐

最大化 tokens/s

```
--max-num-seqs 128
--max-num-batched-tokens 8192
--quantization fp8
--enable-prefix-caching
# Continuous Batching 默认开启
```

Terminal

目标 B: 最低延迟

最小化 TTFT/TPOT

```
--enable-chunked-prefill
--max-num-batched-tokens 512
--speculative-model <eagle_model>
--num-speculative-tokens 4
# 生产环境考虑 PD 分离
```

Terminal

目标 C: 节省显存

解决 OOM 问题

```
--max-model-len 4096
--quantization fp8
--kv-cache-dtype fp8
--max-num-seqs 32
--preemption-mode swap
```

Terminal

典型优化目标与参数组合

常见瓶颈诊断

场景 1: TTFT 过高 (>2s)

Thinking: 如何诊断?

```
if prompt_len > 4096 and num_requests > 10:  
    bottleneck = "Prefill Compute Bound"  
elif kv_cache_usage > 0.9:  
    bottleneck = "Memory Bound (Swapping)"
```

Thinking: Chunked Prefill 为什么有效?

长 prompt 的 Prefill 非常耗时。一次性计算整个 prompt 会独占 GPU，其他 decode 请求被迫等待。拆分成小块后交替执行，就像高速公路的“分时复用” -- 长途车每跑一段就让出车道给短途车通行。

Thinking: 如何解决?

- Chunked Prefill: 长 prompt 拆分 512 tokens，与 decode 交错
- Prefix Caching: 系统 prompt 的 KV Cache 复用
- PD 分离: Prefill 单独部署，不阻塞 Decode

常见瓶颈诊断

场景 2: TPOT 不稳定 (抖动大)

Thinking: 如何诊断?

- batch size 波动大 (1-32) → 请求到达不均匀
- 推测解码接受率骤降 → 模型分布偏移

Thinking: 如何解决?

- `--max-num-seqs 64` 控制 batch 波动上限
- 高温度 (>0.8) 降低 draft length (4→2) 或用 Typical Acceptance
- PD 分离后 Decode 独立运行, TPOT 更稳定

常见瓶颈诊断

场景 3: GPU 利用率低 (<50%)

Thinking: 如何诊断?

- Memory-Bound: seq_len 过长, 显存占满但计算未满
- 通信开销: 张量并行时 NVLink 通信成为瓶颈

Thinking: 如何解决?

- 增大 --max-num-seqs, 直到 GPU Compute 达 85%+
- --quantization fp8 --kv-cache-dtype fp8 省显存提吞吐
- Batch < 8 时启用推测解码 (2-3x 加速)

Thinking: 为什么 Batch Size 大了推测解码效果会减弱?

推测解码的核心收益在于把 Decode 阶段从 memory-bound 变为 compute-bound。

当 Batch Size 已经很大时, GPU 计算已接近饱和 (本身就是 compute-bound), 草稿模型的额外计算反而成了累赘。=> 就像拼车

常见瓶颈诊断

场景 3: GPU 利用率低 (<50%)

Thinking: 如何诊断?

- Memory-Bound: seq_len 过长, 显存占满但计算未满
- 通信开销: 张量并行时 NVLink 通信成为瓶颈

Thinking: 如何解决?

- 增大 --max-num-seqs, 直到 GPU Compute 达 85%+
- --quantization fp8 --kv-cache-dtype fp8 省显存提吞吐
- Batch < 8 时启用推测解码 (2-3x 加速)

Thinking: 为什么 Batch Size 大了推测解码效果会减弱?

推测解码的核心收益在于把 Decode 阶段从 memory-bound 变为 compute-bound。

当 Batch Size 已经很大时, GPU 计算已接近饱和 (本身就是 compute-bound), 草稿模型的额外计算反而成了累赘。=> 就像拼车

vLLM 性能监控实战

vLLM 性能监控实战

实验环境:

- GPU: RTX 4090D (24GB)
- 模型: Qwen3-0.6B (FP16)
- 推理引擎: vLLM 0.18.0
- 启动配置: basic (默认参数)

启动 vLLM 服务

```
# 基础版启动命令 (验证环境可用)
python -m vllm.entrypoints.openai.api_server \
    --model Qwen/Qwen3-0.6B \
    --port 8000 \
    --dtype float16 \
    --enforce-eager \
    --trust-remote-code
```

CASE1: 流式输出与延迟测量 (TTFT / TPOT)

TO DO: 流式输出与延迟测量 (TTFT / TPOT)

通过流式输出 (stream=True) 精确测量每个 token 的到达时间，从而计算 TTFT 和 TPOT 分布。

```
import time
from openai import OpenAI
client = OpenAI(base_url="http://localhost:8000/v1",
api_key="not-needed")
def measure_streaming_latency(prompt, max_tokens=200):
    """通过流式输出精确测量 TTFT 和每个 token 的 TPOT"""
    token_timestamps = []
    request_start = time.perf_counter()
    stream = client.chat.completions.create(
        model="Qwen3-0.6B",
        messages=[{"role": "user", "content": prompt}],
        stream=True, max_tokens=max_tokens
    )
```

```
for chunk in stream:
    if chunk.choices[0].delta.content:
        token_timestamps.append(time.perf_counter())

# TTFT: 从发送请求到收到第一个 token
ttft = (token_timestamps[0] - request_start) * 1000
# TPOT: 相邻 token 间隔
intervals = [(token_timestamps[i] - token_timestamps[i-1]) * 1000
              for i in range(1, len(token_timestamps))]
return ttft, intervals
```

3-流式输出与延迟测量.py

CASE1：流式输出与延迟测量（TTFT / TPOT）

用不同长度的 Prompt 测试，观察 TTFT 随 Prompt 长度的变化（Prefill 计算量越大，TTFT 越高）：

实验结果：TTFT / TPOT 测量

测试场景	TTFT (ms)	TPOT 平均 (ms)	TPOT P50 (ms)	TPOT P95 (ms)	吞吐 (tok/s)
短 Prompt ("你好")	355.5	13.6	14.0	14.8	58.8
中 Prompt (自注意力机制)	40.0	8.8	8.6	10.1	111.8
长 Prompt (PagedAttention 背景 + 问题)	63.2	9.3	9.1	10.5	106.3

Thinking：为什么短 Prompt 的 TTFT 反而最高（355ms）？

这是冷启动效应，即 第一个请求时 GPU CUDA kernel 尚未预热（JIT 编译、内存分配等），导致首次 TTFT 显著偏高。后续请求（中/长 Prompt）已完成预热，TTFT 回落到 40-63ms 的正常水平。

在生产环境中，通常会发送"预热请求"来避免冷启动对首批用户的影响。

CASE2: 并发压测 (Continuous Batching 效果验证)

TO DO: 并发压测 (Continuous Batching 效果验证)

以不同并发度发送 20 个请求，观察吞吐量如何随并发度增长 -- 验证课件中"GPU 处理 1 个请求和 8 个请求耗时差不多，但吞吐翻了 8 倍"的结论。

```
import concurrent.futures, requests, numpy as np

def run_concurrent_benchmark(prompts, concurrency, max_tokens=100):
    """以指定并发度发送请求，收集性能数据"""
    all_results = []
    bench_start = time.perf_counter()
    with concurrent.futures.ThreadPoolExecutor(max_workers=concurrency) as executor:
        futures = [executor.submit(send_chat_request, p, max_tokens) for p in prompts]
        for f in concurrent.futures.as_completed(futures):
            all_results.append(f.result())
    total_time = time.perf_counter() - bench_start
    # 计算 QPS、吞吐量、延迟分位数...
```

4-并发压测.py

CASE2：并发压测（Continuous Batching 效果验证）

实验结果：并发度 vs 性能

并发度	QPS (req/s)	吞吐量 (tok/s)	P50 延迟 (ms)	P99 延迟 (ms)
1	1.02	101.0	899.8	1453.9
2	2.02	201.6	985.1	1048.2
4	3.85	385.4	1018.2	1128.3
8	6.35	635.0	1037.3	1101.5
16	9.87	987.2	1520.6	2085.3

- 吞吐量线性增长：并发从 1 到 8，吞吐量从 101 增长到 635 tok/s（约 6.3 倍），P50 延迟仅从 900ms 增到 1037ms，几乎没增加 -- 这就是 Continuous Batching 的效果
- 拐点出现：并发 16 时，P99 延迟跳升到 2085ms，说明 GPU 算力接近饱和，从 Memory-Bound 转向 Compute-Bound
- 课件验证："处理 1 个请求和 8 个请求耗时差不多" -- P50 延迟从 900ms 到 1037ms 仅增 15%，但吞吐翻了 6 倍

CASE3: Prometheus 指标采集

TO DO: Prometheus 指标采集

vLLM 默认暴露 /metrics 端点（Prometheus 格式），可以实时监控服务状态。以下代码从该端点采集并解析关键指标：

```
import re, requests

def fetch_and_parse_metrics():
    """从 vLLM /metrics 端点采集关键指标"""
    raw = requests.get("http://localhost:8000/metrics").text
    metrics = {}
    # 提取 Counter / Gauge 类型指标
    patterns = {
        "prompt_tokens_total": r'vllm:prompt_tokens_total[^\n]*?\s+([\d.]+)',
        "generation_tokens_total": r'vllm:generation_tokens_total[^\n]*?\s+([\d.]+)',
        "num_requests_running": r'vllm:num_requests_running\s+([\d.]+)',
        "gpu_cache_usage_perc": r'vllm:gpu_cache_usage_perc\s+([\d.]+)',
    }
    for key, pattern in patterns.items():
        match = re.search(pattern, raw)
        if match:
            metrics[key] = float(match.group(1))
    return metrics
```

```
for key, pattern in patterns.items():
    match = re.search(pattern, raw)
    if match:
        metrics[key] = float(match.group(1))
return metrics
```

5-性能指标采集.py

CASE3: Prometheus 指标采集

实验结果：负载前后指标对比

指标	负载前	负载后 (20 个并发请求)	增量
输入 token 累计 (prompt_tokens_total)	6,514	6,774	+260
生成 token 累计 (generation_tokens_total)	21,634	24,634	+3,000

持续监控模式下，可以每隔几秒采集一次指标，实时观察生成 token 数的增长和 KV Cache 使用率变化：

```
# 负载期间持续监控 (每2秒采集一次)
[ 0.1s] 运行: 0 等待: 0 KV Cache: 0.0% 生成tokens: 21634
[ 2.1s] 运行: 0 等待: 0 KV Cache: 0.0% 生成tokens: 22552 # +918
[ 4.1s] 运行: 0 等待: 0 KV Cache: 0.0% 生成tokens: 23450 # +898
[ 6.2s] 运行: 0 等待: 0 KV Cache: 0.0% 生成tokens: 24443 # +993
[ 8.2s] 运行: 0 等待: 0 KV Cache: 0.0% 生成tokens: 24634 # 完成
```

CASE3: Prometheus 指标采集

指标解读:

- 生成速度: 每 2 秒新增约 900 tokens, 即约 450 tok/s 的实际吞吐 (8 并发 x ~56 tok/s/req)
- KV Cache 0%: Qwen3-0.6B 模型很小 (1.12GB), RTX 4090D 有 24GB 显存, KV Cache 占比极低。换成更大的模型 (如 7B/13B) 或更长的上下文, 该指标会显著上升
- 运行中请求数为 0: vLLM v0.18.0 的部分 Gauge 指标在采样间隔之间可能为 0 (请求已在两次采样之间完成)。生产环境可搭配 Grafana 做秒级采集

打卡：vLLM 性能监控



使用vLLM部署大模型，并对性能进行监控，比如：

- 部署 qwen3-0.6B
- 流式输出与延迟测量 (TTFT / TPOT)
- 并发压测 (Continuous Batching 效果验证)
- Prometheus 指标采集

观察输出的结果，对vLLM性能能进行监控和分析



Thank You
Using data to solve problems